



Tecnico Superiore per lo sviluppo di sistemi full stack per web, mobile e desktop
ITS Software Architect Specialist
NUOVA EDIZIONE

KubeX

Simone Cuzzolin

Tecnico Superiore per lo sviluppo di sistemi full stack per web, mobile e desktop
ITS Software Architect Specialist
NUOVA EDIZIONE

Biennio 2023-25

KubeX

Simone Cuzzolin

Indice

Chi sono	1
Vantea Smart	2
Il tirocinio.....	3
Obiettivo	3
Il percorso	3
Le basi di Linux.....	4
Angular	5
Il metodo di lavoro	5
Strumenti utilizzati	6
Competenze maturate	8
Il progetto KubeX	9
Il cliente	9
Glossario	10
Cos'è.....	11
Architettura	11
Componenti custom e librerie esterne	12
Le funzionalità	23
Login e Logout	23
Utenti	27
Profili	32
Le schedulazioni	36
Export	41
I cataloghi.....	43
Meta cataloghi.....	44
Cataloghi.....	51
Conclusione	54
Ringraziamenti	55
Sitografia	56

Chi sono

Mi presento, mi chiamo Simone Cuzzolin e sono nato a Monza il 19 maggio 2002.

Nel 2022 mi sono diplomato in grafica e comunicazione presso l'istituto IIS G. Meroni di Lissone. Durante il mio percorso di studi grafici, ho avuto la possibilità di svolgere un tirocinio curriculare presso un'azienda operativa nell'ambito della grafica pubblicitaria; lì mi sono occupato della creazione di volantini, brochure, roll up e biglietti da visita.

In seguito, ho lavorato nel settore playout presso Sky Italia dove ho migliorato le mie capacità comunicative e di problem solving.

Successivamente al termine degli studi ho iniziato a sviluppare una passione per il mondo della UI/UX Design, la quale mi ha spinto ad iniziare un nuovo percorso.

Nel 2023 mi sono iscritto all'ITS Angelo Rizzoli, più precisamente al corso di Software Architect. Mi sono messo in gioco con l'obiettivo di ampliare le mie conoscenze grafiche e imparare a realizzare un'applicazione dalla fase di disegno su carta fino alla conclusione del progetto.

Durante il corso ho ottenuto una visione chiara dei linguaggi di programmazione e del processo relativo allo sviluppo di un applicativo. Ma solo con il passare del tempo, e approfondendo ogni argomento, mi sono reso conto di essere particolarmente interessato allo sviluppo frontend.

Ho avuto l'occasione di concentrarmi nel campo del frontend soprattutto durante il tirocinio formativo presso l'azienda Vantea Smart. Ho lavorato a distanza nell'area di sviluppo della sede di Roma, dove ho potuto mettere in pratica le competenze precedentemente acquisite e sviluppare ulteriori abilità.

Vantea Smart

Vantea SMART è una società italiana quotata in borsa e presente nel mercato **Euronext Growth Milan**.

Ha sedi operative a Milano, Napoli, Aquila e Roma, ognuna delle quali è specializzata in un ambito differente. La sede di Roma, in cui ho lavorato in questi mesi, si occupa dell'area di sviluppo.

Vantea opera da oltre trent'anni nel settore IT, più precisamente nell'ambito di Cyber Security. I suoi clienti vengono seguiti lungo tutto il percorso di trasformazione digitale da sviluppatori attenti e preparati.

L'azienda offre servizi come:

- **Infrastructure Security**
- **Identity and Access Management**
- **Threat Detection**
- **Security Operations**
- **Security Consulting**

e prodotti software di proprietà quali:

- **KubeX**
- **Infosync**

Il tirocinio

Obiettivo

L'obiettivo principale del tirocinio curricolare è stato quello di formarmi come frontend developer. Ho svolto attività di sviluppo e analisi del codice, ho appreso nuove tecniche di sviluppo e ho imparato ad utilizzare nuove tecnologie.

Con gli obiettivi chiari in mente fin dall'inizio, ho raggiunto la fine del tirocinio pienamente soddisfatto dell'esperienza e delle conoscenze di sviluppo acquisite in ambito frontend.

Il percorso

Il percorso di tirocinio è iniziato il diciannove gennaio presso la sede di Milano.

Le prime settimane del percorso le ho svolte in comune con alcuni ragazzi iscritti ad altri percorsi formativi di ITS Rizzoli.

Durante questo periodo siamo stati formati dalla sede di Milano, che opera nell'ambito IAM, ovvero Identity Access and API Management.

Dopo due settimane di stage mi sono distaccato dal gruppo di Milano per dedicarmi completamente allo studio del framework di Google, ovvero Angular.

Inizialmente ho seguito dei corsi su YouTube per migliorare la mia conoscenza generale sul framework, ma nel frattempo mi sono allenato realizzando dei mini-progetti per applicare le funzionalità imparate nei video. Inoltre, ho sviluppato un piccolo progetto personale contenente tutte le nozioni imparate durante la visione dei corsi.

Una volta terminata la formazione, mi sono affiancato ad un ragazzo della sede di Roma per imparare come lavorare tramite Angular a livello professionale.

Inizialmente il mio lavoro veniva revisionato dal tutor o svolgevamo delle sessioni di pair-coding, così da lavorare insieme sul codice ed imparare direttamente sul campo.

Una volta presa confidenza e dimestichezza con il framework, ho iniziato a lavorare sul progetto KubeX.

Ho iniziato sempre tramite sessioni di pair-coding con un collega e successivamente ho continuato a lavorare al codice in completa autonomia, senza riscontrare difficoltà nella stesura del codice e nella risoluzione degli errori.

Le basi di Linux

Nel corso delle prime settimane del mio percorso formativo, ho approfondito le basi del sistema operativo Linux insieme ai colleghi dell'area IAM. Durante questa fase iniziale ci siamo concentrati sullo studio dell'ambiente Linux, della linea di comando e dei principali comandi della shell.

Struttura ad albero delle directory di Linux

Una parte importante del percorso è stata la comprensione della struttura ad albero delle directory nel sistema operativo Linux. Di seguito, una panoramica delle principali directory.

Directory	Descrizione
/	Directory principale, radice di tutto il filesystem.
/bin	Comandi essenziali per tutti gli utenti.
/boot	File per l'avvio del sistema (kernel, GRUB).
/dev	File di dispositivo per hardware e periferiche.
/etc	File di configurazione del sistema (es. rete, servizi).
/home	Home directory degli utenti.
/lib	Librerie condivise essenziali per i binari in /bin e /sbin.
/mnt	Punto di montaggio temporaneo per dischi o filesystem.
/proc	File virtuali che mostrano informazioni sul sistema e i processi.
/root	Home directory dell'utente amministratore root.
/sbin	Comandi di sistema essenziali per amministratori.
/tmp	File temporanei, spesso eliminati al riavvio.
/usr	Programmi e librerie per gli utenti.
/var	File variabili come log di sistema, spool di stampa, file temporanei estesi.

Comandi principali usati

Durante le esercitazioni ho utilizzato numerosi comandi della shell. Eccone alcuni.

Comando	Descrizione
find	Comando usato per eseguire la ricerca di un file.
cat	Comando usato per stampare il contenuto di un file.
tac	Comando usato per stampare il contenuto di un file in ordine di righe inverso.
sort	Comando usato per ordinare i file.
ls	Comando usato per elencare i file e le directory presenti nella directory corrente.
netstat	Comando usato per visualizzare le informazioni relative alla rete.
cut	Comando usato per estrarre porzioni di testo dalle righe di un file.
ping	Comando usato per verificare se un host è raggiungibile e per misurare il tempo di risposta.
vi	Comando usato per aprire un editor di testo.
grep	Comando usato per cercare del testo all'interno di un file.

awk	Comando usato per leggere ed analizzare le righe di un file ed eseguire azioni specifiche su di esse.
chown	Comando usato per cambiare il proprietario di un file.
chmod	Comando usato per modificare i permessi di accesso a file e directory.
tail	Comando usato per visualizzare le ultime righe di un file.
head	Comando usato per visualizzare le prime righe di un file.
mv	Comando usato per spostare o rinominare file o directory

Angular

Angular è un framework open source, sviluppato da Google, dedicato allo sviluppo di applicazioni web. È scritto in TypeScript e la sua caratteristica principale è permettere agli sviluppatori di costruire Single Page Applications, cioè applicazioni che caricano una singola pagina HTML e aggiornano il contenuto dinamicamente senza dover ricaricare l'intera pagina.

Angular si basa su un'architettura a componenti, in cui ogni parte dell'interfaccia utente è rappresentata da un componente riutilizzabile. Ogni componente è composto da:

- Un file HTML (template) che definisce la struttura;
- Un file CSS/SCSS che ne definisce lo stile;
- Un file TypeScript che gestisce la logica e i dati.

Alcune delle funzionalità principali offerte da Angular sono:

- **Data Binding:** sincronizzazione automatica tra TypeScript e HTML;
- **Dependency Injection:** gestione delle dipendenze tra i vari componenti e servizi;
- **Routing:** navigazione tra diverse pagine e sezioni dell'app senza ricaricare l'intera pagina;
- **Servizi:** gestione della logica condivisa e comunicare con API esterne o backend.

Il metodo di lavoro

All'inizio della mia esperienza ho lavorato su una serie di task assegnati da un collega, con l'obiettivo di applicare in modo pratico le nozioni apprese durante i corsi di formazione. Successivamente, con l'avvio del progetto finale, abbiamo adottato una modalità di lavoro basata su task condivisi: io e il mio collega stabilivamo insieme le funzionalità e le pagine da sviluppare, organizzando le attività in coppia e gestendo in autonomia tempi e priorità.

Negli ultimi due mesi del percorso abbiamo introdotto la metodologia Agile. A differenza delle metodologie tradizionali, che prevedono una pianificazione dettagliata all'inizio del progetto, Agile suddivide il lavoro in cicli chiamati sprint, che possono durare da una a quattro settimane.

Ogni sprint ha obiettivi chiari e si conclude con una revisione del lavoro svolto, permettendo così al team di adattarsi rapidamente a nuove esigenze e modifiche.

Strumenti utilizzati

Durante questo percorso ho utilizzato numerosi strumenti, la maggior parte dei quali erano nuovi per me, ciascuno con una funzionalità diversa dagli altri. Alcuni li ho usati per scrivere codice, mentre altri per registrare la documentazione o per gestire il flusso di lavoro.

WebStorm

WebStorm è un IDE sviluppato da JetBrains, progettato per lo sviluppo web.

Supporta linguaggi come JavaScript, TypeScript, HTML e CSS ed è compatibile con i principali framework frontend, come Angular, React e Vue.js.

Durante il mio percorso di stage l'ho utilizzato come IDE per lo sviluppo frontend, impiegando Angular come framework di sviluppo.

Ho sfruttato molte delle sue funzionalità, come la possibilità di installare plug-in per facilitare lo sviluppo e migliorare la produttività. Inoltre, mi ha permesso di creare sessioni di pair coding in cui è possibile invitare altri sviluppatori a lavorare sul proprio codice in tempo reale, così da poter essere perfettamente sincronizzati.

Microsoft Teams

Microsoft Teams è una piattaforma di comunicazione e collaborazione sviluppata da Microsoft, dedicata al lavoro di gruppo.

Offre funzionalità di chat, videochiamate e condivisione file. Inoltre, consente di organizzare riunioni e creare gruppi di collaborazione per lavorare tra colleghi.

Ho utilizzato questo software giornalmente per comunicare e collaborare con il team di cui facevo parte. Ho sfruttato principalmente le funzioni di chat di gruppo, videochiamate, creazione e organizzazione eventi.

Quest'ultima l'ho sfruttata per fissare daily call con il team di sviluppo, grazie alle quali facevamo il punto della situazione sul progetto e comunicavamo con il reparto backend.

Jira

Atlassian Jira è uno strumento dedicato all'organizzazione e alla collaborazione tra teams, sviluppato da Atlassian.

Le principali caratteristiche offerte da Jira sono:

- Eccellenza nel monitoraggio dei problemi, che consente ai teams di stabilire priorità e risolvere le criticità nel modo più efficace;
- Ottimo supporto per la gestione dei progetti tramite metodologia Agile;
- Capacità di integrarsi con strumenti, e software, esterni all'ecosistema Atlassian; ad esempio, estensioni e plug-in per i principali software dedicati allo sviluppo, come Visual Studio Code e gli applicativi di JetBrains.

Jira offre numerose funzionalità per la pianificazione degli sprint, consentendo ai teams di gestire il lavoro in modo efficace. Tra queste vi sono la mappatura di storie utente, la pianificazione degli sprint e la gestione dei backlog.

Bitbucket

Bitbucket è uno strumento dedicato alla gestione del versionamento del codice tramite Git, sviluppato da Atlassian.

Come altri strumenti, quali GitHub o GitLab, consente di creare repository in cui gli sviluppatori caricano il codice e lo gestiscono all'interno del team.

Come altri VCS (Version Control System), offre la possibilità di svolgere operazioni sul codice, come:

- **Pull:** scaricare in locale il codice presente nel repository remoto;
- **Push:** caricare le modifiche effettuate in locale sul repository remoto;
- **Merge:** unire le modifiche al codice già presente nel repository.

Durante lo stage, ho utilizzato Bitbucket come VCS, sfruttandolo per gestire il codice all'interno del team di sviluppo. Ho creato branch dedicati a ciascuna delle implementazioni da realizzare, così da organizzare al meglio il flusso di lavoro.

Confluence

Atlassian Confluence è uno strumento dedicato alla collaborazione e alla condivisione di informazioni, sviluppato da Atlassian.

Confluence è pensato per favorire la comunicazione efficace e la gestione delle conoscenze all'interno del team.

È strutturato in spazi, ovvero aree specifiche dedicate a gruppi o progetti. Ogni spazio è dotato di una dashboard che fornisce informazioni relative ad esso.

Durante il mio percorso ho utilizzato Confluence per la stesura della documentazione relativa al progetto KubeX e come lavagna digitale per annotare idee e implementazioni emerse durante le varie riunioni tra i teams di sviluppo.

OpenVPN

OpenVPN è un protocollo open source per creare connessioni VPN sicure, basato su standard crittografici di grande affidabilità.

Durante il mio periodo di stage ho utilizzato l'applicazione OpenVPN, che consente di configurare e personalizzare server VPN oppure di connettersi a server già esistenti.

Competenze maturate

Durante il percorso ho maturato e acquisito molte competenze, quali:

Hard Skills

- Sviluppo di Single Page Applications (SPA);
- Realizzazione di progetti personali e aziendali con Angular;
- Autonomia nello sviluppo e debugging;
- Conoscenza IDE WebStorm;
- Utilizzo Git e VCS;
- Gestione progetti;
- Creazione documentazione;
- Utilizzo di OpenVPN.
- Linguaggi e tecnologie web:
 - HTML, SCSS e TypeScript;
 - Sviluppo con framework Angular.
- Sistemi Linux:
 - Navigazione e gestione tramite terminale;
 - Conoscenza della struttura ad albero del filesystem Linux;
 - Utilizzo dei principali comandi Linux;
 - Utilizzo editor vi.
- Metodologia Agile:
 - Organizzazione del lavoro in sprint;
 - Pianificazione e revisione periodica degli obiettivi;
 - Partecipazione attiva a daily call.

Soft Skills

- Team working e comunicazione interfunzionale;
- Problem solving in contesti collaborativi;
- Capacità di apprendimento autonomo (formazione online su Angular);
- Gestione autonoma delle attività;
- Adattabilità e flessibilità.

Il progetto KubeX

Il cliente

Il cliente che utilizza il software KubeX è Telecom Italia.

Nel 2022, Vantea ha acquisito il software KubeX da Kataskopeo S.r.l., entrando così nel mercato dell'antifrode.

Attualmente, Vantea si occupa di mantenere il software aggiornato, continuando a gestirlo con la professionalità del proprio team di sviluppatori.

La richiesta di Telecom era di disporre di un software che consentisse di effettuare analisi dinamiche sui dati e offrisse funzionalità per creare correlazioni libere; tutto questo senza dover richiedere un'implementazione da parte degli sviluppatori ogni volta che si presentasse questa esigenza.

I bisogni e gli obiettivi del progetto sono stati definiti in costante e stretto contatto con l'utente finale del software, supportandolo durante le analisi e comprendendo di volta in volta le sue necessità.

Telecom richiede periodicamente nuovi requisiti che il software deve soddisfare.

Questi requisiti possono riguardare aspetti diversi, come l'acquisizione di nuovi dati nel database, con o senza logiche di business, da mettere a disposizione direttamente sul software; oppure la creazione di nuove modalità per interrogare e visualizzare i dati all'interno del software.

I tempi di realizzazione della prima versione del software, sviluppata da Vantea, sono stati di sei mesi. La versione attuale, su cui ho avuto modo di lavorare, è ancora in fase di sviluppo.

La primissima versione del software, precedente all'acquisizione da parte di Vantea, era scritta in un linguaggio completamente diverso, con molti limiti e vulnerabilità. Su queste basi è stato creato un progetto moderno, che ha adottato sia tecnologie e metodologie più avanzate.

La realizzazione della nuova versione nasce da un mix di esigenze: rendere il software più sicuro e facilitare la gestione delle continue richieste da parte del cliente.

Attualmente l'inserimento di nuovi dati nel software non è più affidato agli sviluppatori, ma è diventata una semplice configurazione a disposizione del cliente, che può così usufruirne direttamente.

Glossario

Di seguito i termini e concetti che verranno utilizzati nelle spiegazioni delle varie funzionalità.

Meta catalogo

I meta cataloghi sono raccolte di metadati relativi a un database al quale è stata effettuata una connessione.

Essi contengono tutte le tabelle e le viste presenti nel database, ma solo come metadati e in formato query, come se fossero uno screenshot del database.

Catalogo

I cataloghi sono raccolte di query eseguite su un meta catalogo.

Possiamo quindi dire che sono raccolte di viste riorganizzabili in modo diverso, per soddisfare esigenze specifiche.

Viste

Le viste sono le query eseguite sul database.

Regole

Le regole sono query e si possono dividere in:

- Regole di meta catalogo
- Regole di catalogo
- Regole utente

Materializzazione

Il concetto di materializzazione consiste nella creazione di una materializzata, ovvero una tabella temporanea che contiene un sottoinsieme (subset) dei risultati di una regola utente.

La materializzazione è asincrona perché, se la regola utente è complessa o la tabella da cui si estrae è molto grande, i tempi di risposta potrebbero essere troppo lunghi, consentendo così all'utente di non dover rimanere vincolato alla pagina fino al completamento dell'estrazione.

Una volta ottenuto il risultato della materializzata, per visualizzarlo sarà necessario creare una nuova regola utente e selezionarlo.

Cos'è

Kubex è un'applicazione che permette di eseguire analisi sui dati senza necessitare di conoscenze approfondite di query, ma solo una conoscenza generale del settore, poiché consente di effettuare analisi con semplici passaggi, senza dover scrivere alcuna riga di codice.

Kubex offre il vantaggio di fornire funzionalità che permettono facilmente all'amministratore di collegarsi a qualsiasi database e di creare nuovi oggetti da mettere a disposizione degli utenti, senza dover sviluppare modifiche nel frontend o nel backend.

Questi oggetti possono essere assegnati a utenti diversi in base al loro profilo.

Le principali funzionalità offerte dal software sono:

- Creazione di cataloghi da mettere a disposizione di specifici profili;
- Creazione di regole utente;
- Funzionalità di analisi;
- Funzionalità di reportistica (grafici statici, ovvero "istantanee" dei dati);
- Funzionalità di dashboard, con grafici dinamici che si aggiornano automaticamente in base ai dati presenti nel database;
- Funzionalità di schedulazione: possibilità di programmare l'esecuzione di regole utente, come esportazioni in Excel, creazione di materializzate, generazione di report, e invio dei risultati via e-mail;
- Funzionalità di importazione dati utente nel catalogo: importazione di file Excel, CSV, TXT nel database del catalogo; l'oggetto viene mappato automaticamente, consentendo l'utilizzo dei dati importati con tutte le funzionalità disponibili.

Architettura

Il sistema è implementato attraverso un'architettura a microservizi, con frontend SPA realizzato in Angular.

Tale architettura consente di massimizzare la resilienza del sistema quando correttamente implementata e deployata, rendendo le piattaforme che la adottano più affidabili e fruibili.

Tutti i microservizi sono sviluppati utilizzando Spring Boot su JVM 11 ed espongono servizi REST.

Lo strato di accesso ai dati è gestito tramite Spring Data Repositories (JPA/Hibernate), che consente l'utilizzo sia di database Postgres, sia di numerosi altri database compatibili con queste tecnologie.

Principali microservizi:

API Gateway

Componente architetturale che espone e orchestra i servizi REST interni al cluster; costituisce l'unico punto di accesso ai servizi.

IAM

Identity Access Management, fornisce servizi di autenticazione, autorizzazione e gestione amministrativa dei profili autorizzativi.

Query

Memorizza i metadati relativi ai cataloghi (connessioni, tipologia e struttura dei database, mappatura logica, ecc.) e implementa la generazione delle interrogazioni verso le basi dati del cliente. È in grado di interfacciarsi con circa 25 database, tra cui i principali prodotti commerciali (Oracle, SQL Server, ecc.) e alcune soluzioni cloud.

Export

Fornisce servizi per la generazione di esportazioni dei dati.

Docs

Gestisce i servizi relativi alla documentazione.

Message Broker

Costituisce il substrato per la comunicazione asincrona tra i microservizi.

Componenti custom e librerie esterne

Durante lo sviluppo del progetto ci siamo appoggiati a un template esterno, Fuse Theme.

Fuse è un template dedicato allo sviluppo web che offre numerose soluzioni utili per il progetto, come componenti personalizzati, set di icone, animazioni, layout, supporto alla multilingua tramite la libreria dedicata Transloco, e un'implementazione per l'autorizzazione tramite JWT.

Abbiamo utilizzato Fuse come scheletro del nostro progetto, sfruttando i layout e i componenti messi a disposizione.

Nel corso dello sviluppo, ci siamo trovati a dover creare componenti custom da riutilizzare più volte all'interno del progetto.

In particolare, è stata necessaria la creazione di componenti versatili, adatti a essere impiegati in diversi contesti.

Di seguito, i principali componenti custom realizzati da noi.

Data table

Data Table, come suggerisce il nome, è un componente che abbiamo creato per rappresentare tabelle più o meno complesse.

Per essere utilizzato, il componente necessita di tre oggetti in input, di cui uno opzionale:

Config

Come indica il nome, è la configurazione della struttura della tabella. Ci aspettiamo un oggetto conforme all'interfaccia riportata di seguito.

```
export interface IDdataTableConfig {
  dataMode: 'CLIENT-SIDE' | 'SERVER-PAGINATED';
  sortAndFilterMode: 'SINGLE_SORT' | 'MULTI_SORT_AND_FILTER';
  columnMode: ColumnMode;
  selectionType?: SelectionType;
  headerHeight: number;
  footerHeight: number;
  rowHeight: number;
  columnWidth?: number;
  scrollbarV?: boolean;
  scrollbarH?: boolean;
  colorClass?: string;
  columnResizable?: boolean;
  queryLimit: number;
  showLocalSearchInput?: boolean;
  showRowDetailContextMenu?: boolean;
  reloadOnChangeLanguage?: boolean;
}
```

ApiCallback

È una callback in cui vengono definite le righe e le colonne della tabella, in modo che il componente possa riconoscerle e gestirle correttamente.

```
export interface IDdataTableData {
  rows: IDdataTableRow[];
  columns: IDdataTableColumn[];
  totalCount: number;
  refreshColumns?: boolean;
  refreshRows?: boolean;
  footerMessage?: string;
}

export interface IDdataTableRow {
  rowItem: IDdataTableRowItem;
  rowActions?: IDdataTableAction[];
  rowCellActions?: IDdataTableRowCellAction;
}
```

```

export interface IDataTableRowItem {
  [propName: string]: string | number | boolean | IDataTableIconValue |
  DataTableValueSpinner;
}

export interface IDataTableAction {
  id: string;
  icon: string;
  tooltip: string;
}

export interface IDataTableRowCellAction {
  [propName: string]: IDataTableColumnActions[];
}

export interface IDataTableColumnActions {
  idColumn: string;
  actions: IDataTableAction[];
}

export interface IDataTableColumn extends TableColumn {
  type: Type;
  flexGrow?: number;
  columnActions?: IDataTableAction[];
  cellActions?: IDataTableAction[];
  hidden?: boolean;
  [propName: string]: any;
}

```

ColumnActions

Questo oggetto è opzionale e serve a definire, se necessario, le azioni eseguibili sulle varie righe della tabella.

```

export interface IDataTableColumnActions {
  idColumn: string;
  actions: IDataTableAction[];
}

```

Il Componente:

name	active	cronExpression	created	lastModified	ruleName	nextExecutionDate	
ABC	ABC	ABC	📅	📅	ABC	📅	
prova schedulazione simone	🚫	Alle 12:20, ogni giorno	24/04/2025 16:17:56	16/05/2025 18:19:04	calo	18/06/2025 12:20:00	👍 📄 🔄 🗑️ ⚙️
test simone	🚫	Alle 10:10, ogni giorno	05/10/2023 10:09:59	20/05/2025 11:24:10	second	18/06/2025 10:10:00	👍 📄 🔄 🗑️ ⚙️
test 21	🚫	Alle 11:12, ogni giorno	01/12/2023 10:11:52	16/05/2025 14:51:16	test simone	18/06/2025 11:12:00	👍 📄 🔄 🗑️ ⚙️
TEST DEB	🚫	Alle 15:30, ogni giorno	20/05/2025 15:29:41	26/05/2025 10:30:04	OL FISSO 2	18/06/2025 15:30:00	👍 📄 🔄 🗑️ ⚙️
ciao	✅	Alle 15:19, ogni giorno	03/04/2025 15:15:02	26/05/2025 10:34:19	calo	18/06/2025 15:19:00	👍 📄 🔄 🗑️ ⚙️

Una volta renderizzato, il componente apparirà come mostrato nell'immagine soprastante, visualizzando una tabella interattiva con funzionalità di gestione, filtraggio e azioni per riga (se presenti).

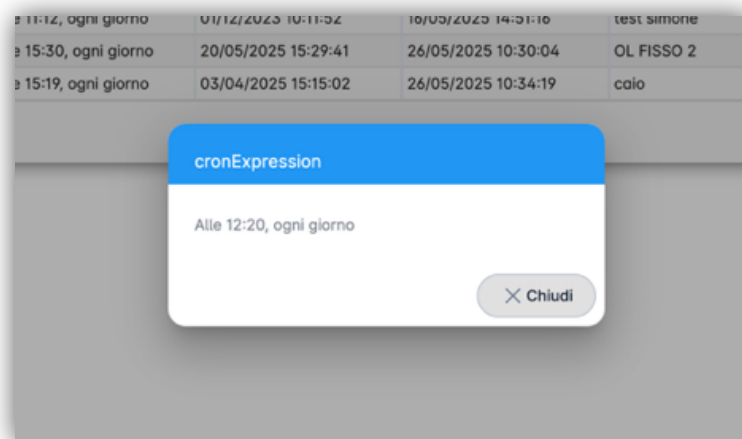
La tabella è composta da tre sezioni: header, body e footer.

Header: contiene le informazioni relative alle colonne, come il nome della colonna, l'icona associata e l'indicazione se la colonna è ordinabile o meno.

Body: mostra le righe della tabella con le informazioni corrispondenti e, se presenti, le azioni applicabili su ciascuna riga.

Footer: fornisce informazioni sulla pagina corrente della tabella, come il numero totale di righe e un elemento di navigazione per spostarsi tra le pagine.

Cliccando su una cella, è possibile visualizzare il valore contenuto in essa tramite un dialog, realizzato in un componente separato che utilizza il dialog fornito da Angular Material.



Cliccando sull'intestazione di una colonna, se sono presenti le frecce, sarà possibile ordinare le righe della tabella: in ordine alfabetico per stringhe, in ordine crescente o decrescente per numeri, e in ordine di data o data/ora per i campi temporali.

Quando viene cliccato un pulsante di azione su una riga (posti a fine riga, se presenti), il componente emette un output contenente i dati relativi alla riga e all'azione selezionata, permettendo al componente esterno che ha implementato la data table di gestire l'evento di click.

KxSearchInput

KxSearchInput è un componente che abbiamo creato per soddisfare l'esigenza di filtrare un array di stringhe senza dover implementare ogni volta la logica di ricerca.

Il componente riceve in input due oggetti:

Config

- Contiene un oggetto di tipo IKxSearchInputConfig, il quale definisce proprietà come:
- L'altezza del componente
- La presenza o meno di un pulsante per avviare la ricerca
- Il testo del placeholder dell'input
- Eventuale classe CSS custom da applicare
- Il nome della chiave dell'oggetto sulla quale eseguire la ricerca

Rows

Contiene un oggetto di tipo IKxSearchInputRow, ovvero un oggetto che rappresenta i dati da filtrare.

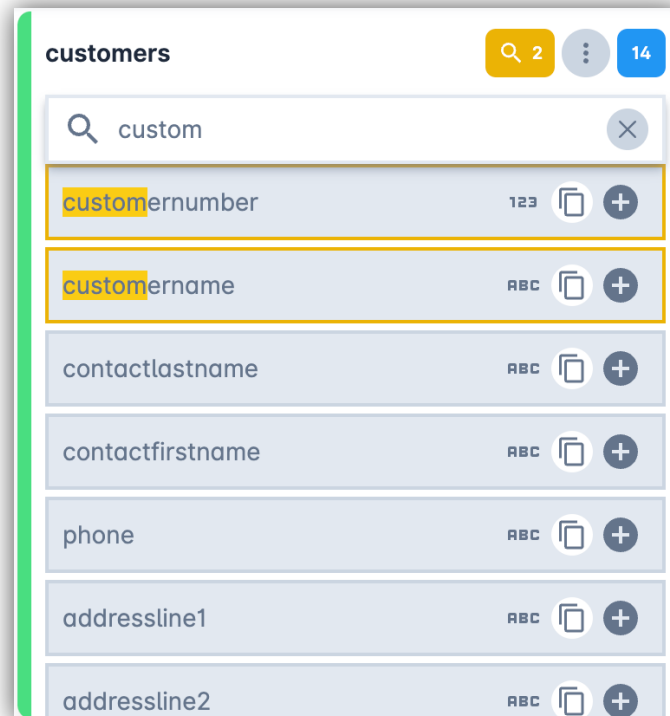
```
export interface IKxSearchInputConfig {  
  height: number;  
  showSearchButton?: boolean;  
  placeholder?: string;  
  wrapperCssClass?: string;  
  properties?: string[];  
}  
  
export interface IKxSearchInputRow {  
  matchInSearch?: boolean;  
  [propName: string]: any;  
}  
  
export interface IKxSearchInputOutput {  
  rows: IKxSearchInputRow[];  
  searchText: string;  
}
```

Il componente emette in output un oggetto contenente:

- Un array con gli elementi filtrati
- Il searchText, ossia la stringa utilizzata per effettuare la ricerca



A livello visivo, il componente appare come un classico campo di input testuale. Nell'immagine seguente è possibile vedere come filtra l'array sottostante. Inoltre, grazie all'output emesso, abbiamo implementato la possibilità di evidenziare nel risultato la stringa trovata all'interno dell'array.



Accordion navigator

L'ultimo componente custom che abbiamo sviluppato è stato anche il più complesso, poiché ha richiesto numerose modifiche e implementazioni per renderlo il più versatile possibile.

Accordion Navigator è un componente creato per visualizzare un array di oggetti, i quali possono contenere a loro volta altri array di oggetti annidati, permettendo di eseguire azioni sia sui singoli elementi sia su gruppi di elementi appartenenti alla stessa struttura.

In input il componente si aspetta un oggetto contenente la configurazione necessaria alla sua creazione:

Config

L'oggetto Config contiene un array di IAccordionConfig.

Ogni IAccordionConfig è composto da:

- Un titolo
- Il totale degli elementi contenuti
- Eventuali filtri da applicare
- Categories, ossia un array di oggetti di tipo category

```
export interface IAccordionConfig {
  title?: string;
  total?: number;
  filters?: AccordionNavigatorFilters[];
```

```

    categories: IAccordionNavigatorCategory[];
}

export enum AccordionNavigatorFilters {
    DATETIME = 'DATETIME',
    STRING = 'STRING',
    BOOLEAN = 'BOOLEAN',
    DATE = 'DATE',
    NUMBER = 'NUMBER',
    DATE_DATETIME = 'DATE_DATETIME',
}

export interface IAccordionNavigatorCategory {
    category: string;
    folders: IAccordionNavigatorFolder[];
    length: number;
    hidden?: boolean;
}

export interface IAccordionNavigatorFolder {
    name: string;
    id?: string;
    items?: IAccordionNavigatorItem[];
    tags?: ICatalogTags;
    color?: string;
    folderIcons?: IAccordionIcon[];
    folderMenuIcons?: IAccordionMenu;
    isSelected?: boolean;
}

export interface IAccordionNavigatorItem {
    name: string;
    id?: string;
    type?: Type;
    itemIcon?: IAccordionIcon[];
    itemMenuIcons?: IAccordionMenu;
    isSelected?: boolean;
    matchInSearch?: boolean;
    matchInFilter?: boolean;
}

export interface ICatalogTags {
    category: string[];
    features?: string[];
}

export interface IAccordionMenu {
    id: string;
    tooltip: string;
    icon: string;
    disabled?: boolean;
    actions: IAccordionIcon[];
}

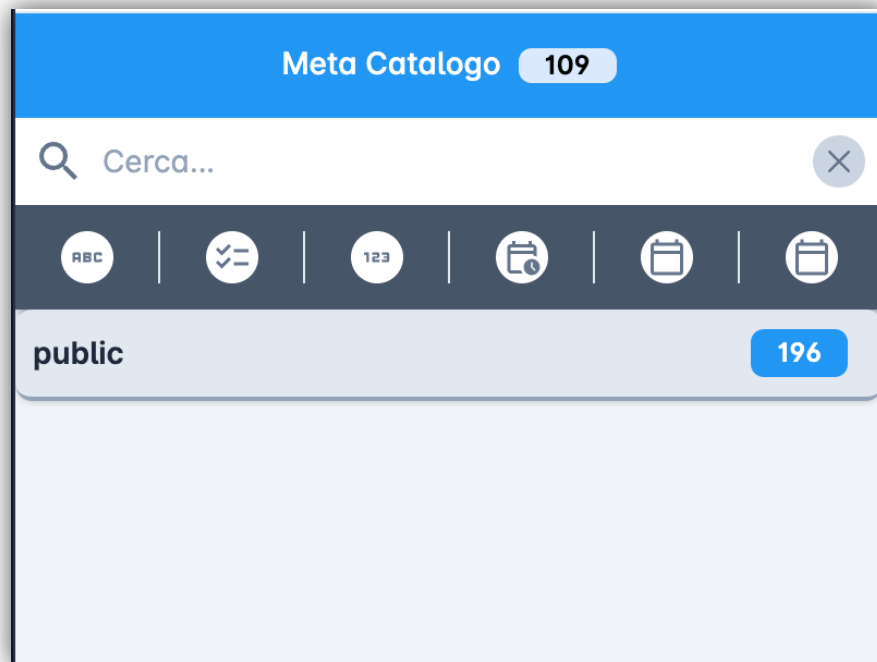
export interface IAccordionIcon {
    id: string;
    icon: string;
}

```



```
tooltip: string;  
disabled?: boolean;  
}
```

Nell'immagine sottostante è possibile vedere come appare a video il componente.

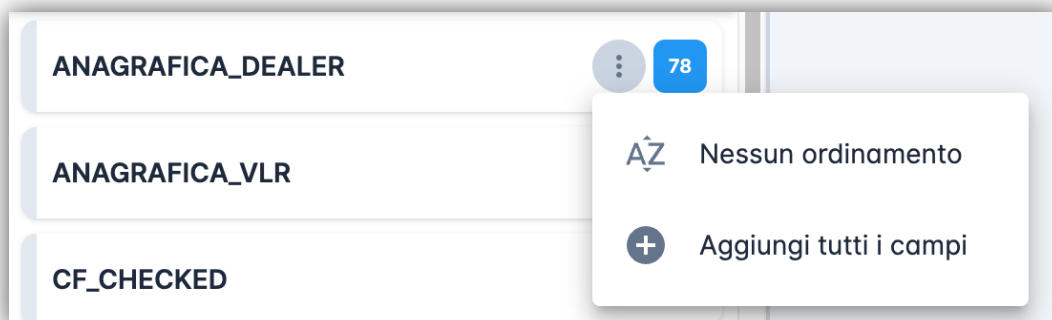


Nella parte superiore troviamo il tab che indica quale accordion è attualmente attivo, mostrando il nome e il numero di elementi presenti al suo interno.

Subito sotto il tab è presente il componente KxSearchInput, utilizzato per filtrare gli elementi visibili nella sezione sottostante, insieme a eventuali filtri specifici basati sul tipo di contenuto.

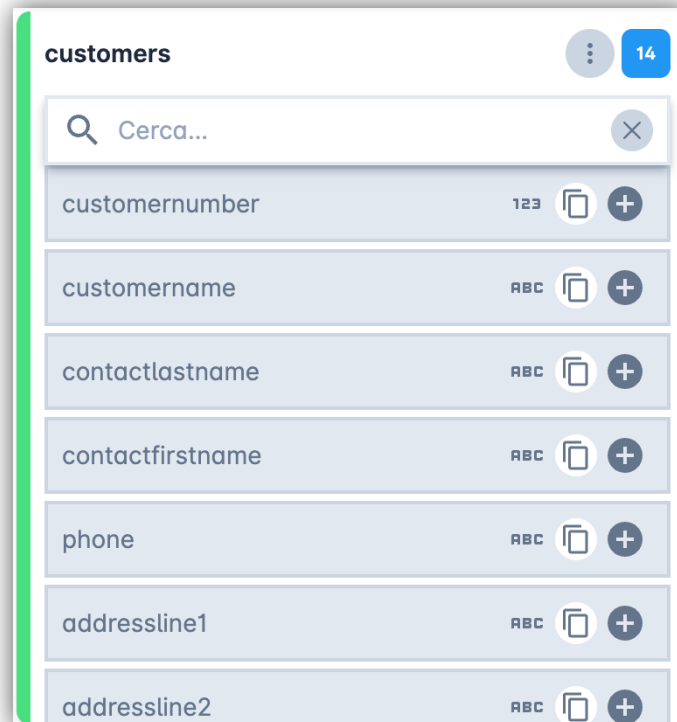
La sezione inferiore presenta le categories, rappresentate come cartelle, ognuna delle quali può contenere altre cartelle (denominate folder), che a loro volta contengono un array di item.

Accanto al nome di ciascuna category viene visualizzato un numero, che indica il totale degli item contenuti nelle folder al suo interno.



Ogni folder dispone di un menu contenente le azioni eseguibili sugli item presenti nella singola folder. Le folder hanno inoltre alcune caratteristiche visive:

- Un bordo colorato sul lato sinistro, se specificato nell'oggetto di input
- Un indicatore numerico sul lato destro, che mostra il numero di item contenuti



Aperto una folder si visualizzano gli item al suo interno. Ciascun item mostra il tipo del suo contenuto e può prevedere azioni specifiche eseguibili dall'utente.

Le azioni effettuate su un singolo item o su una folder vengono comunicate all'esterno tramite specifici eventi di output, che differenziano se l'azione è avvenuta su un item o su una folder. In questo modo, il componente padre che utilizza l'Accordion Navigator può gestire al meglio le interazioni.

L'output relativo al click su una folder è definito dall'interfaccia `IAccordionNavigatorFolderOutput`. Esso include:

- La category nella quale è avvenuta l'interazione
- L'indice della category
- L'indice del tab attivo al momento del click
- E, grazie all'estensione dell'interfaccia `IAccordionFolderOutput`, anche informazioni sulla folder stessa, sull'icona cliccata e sull'indice della folder

Per quanto riguarda il click su un item, l'output è definito dall'interfaccia `IAccordionNavigatorItemOutput`, che estende `IAccordionItemOutput`, a sua volta estensione di `IAccordionFolderOutput`. Contiene tutti gli elementi già elencati sopra, con in più:

- l'item cliccato
- il suo indice
- l'icona selezionata

```

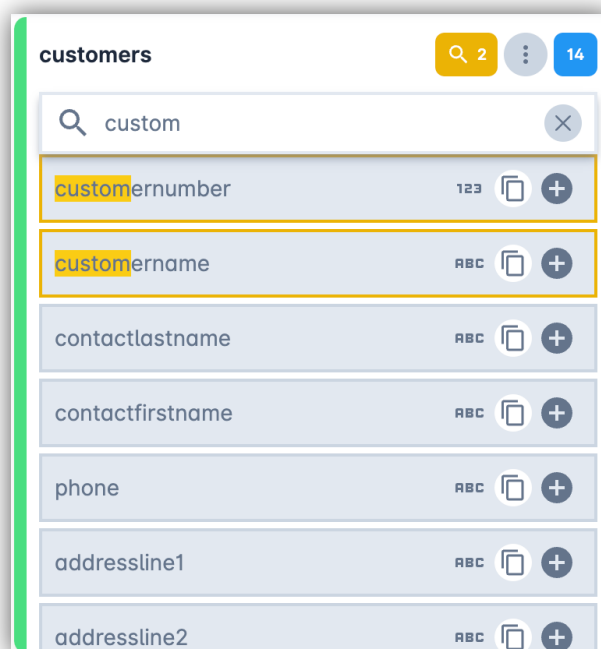
export interface IAccordionNavigatorFolderOutput extends IAccordionFolderOutput {
  category: IAccordionNavigatorCategory;
  categoryIndex: number;
  accordionConfigIndex: number;
}

export interface IAccordionNavigatorItemOutput extends IAccordionItemOutput {
  category: IAccordionNavigatorCategory;
  categoryIndex: number;
  accordionConfigIndex: number;
}

export interface IAccordionFolderOutput {
  folder: IAccordionNavigatorFolder;
  folderIcon?: IAccordionIcon;
  folderIndex?: number;
}

export interface IAccordionItemOutput extends IAccordionFolderOutput {
  item: IAccordionNavigatorItem;
  itemIcon?: IAccordionIcon;
  itemIndex?: number;
}

```



Quando viene eseguita una ricerca testuale, a video vengono mostrati:

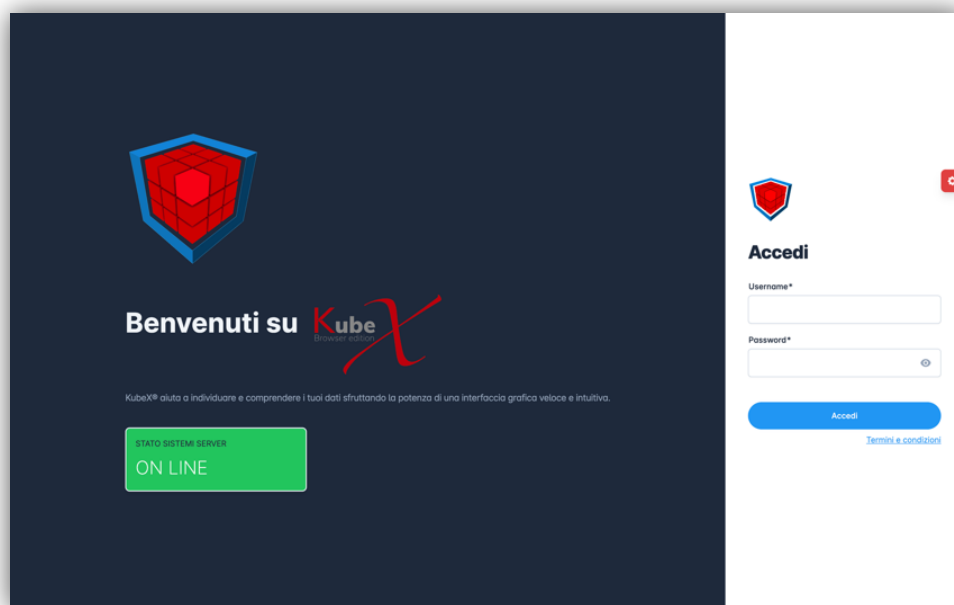
- La stringa cercata evidenziata all'interno degli item in cui viene trovata
- Un bordo colorato sull'item per distinguerlo dagli altri
- Il numero totale di item in cui è stata trovata la stringa cercata

Questa funzionalità è resa possibile grazie all'output fornito dal componente KxSearchInput, integrato all'interno dell'Accordion Navigator.

Le funzionalità

Login e Logout

La pagina di login è realizzata tramite il componente SignInComponent. Il componente gestisce la fase di autenticazione dell'utente, collegandosi al backend tramite un'API all'interno di un servizio.



Per controllare che i dati di accesso siano corretti e presenti all'interno del database, interroghiamo il backend tramite una chiamata API.

```
signIn(): void {  
  this.alert.message = null;  
  this._authService  
    .signIn({  
      username: this.signInForm.get('username').value,  
      password: this.signInForm.get('password').value,  
    })  
    .subscribe({  
      next: (response) => {  
        console.log(response, 'response sign in');  
        this._router.navigate([this.returnUrl]);  
      },  
      error: (err) => {  
        this.alert.message = this._translocoService.translate(  
          'HTTP_ERROR.' + err.error.errorCodes[0]  
        );  
      }  
    })  
}
```

```

    },
  });
}

// Servizio AuthService
set accessToken(token: string) {
  sessionStorage.setItem(ACCESS_TOKEN_STORAGE_KEY, token);
}

get accessToken(): string {
  return sessionStorage.getItem(ACCESS_TOKEN_STORAGE_KEY) ?? '';
}

signIn(credentials: {
  username: string;
  password: string;
}): Observable<any> {

  if (this._authenticated) {
    return throwError('User is already logged in.');
```

```

  }

  return this._authApiService
    .signIn(credentials.username, credentials.password)
    .pipe(
      switchMap((response: { idToken: string }) => {
        if (response.idToken) {
          console.log(response, 'response');
```

```

          this.accessToken = response.idToken;

          this._authenticated = true;

          return this.getCurrentUser();
        } else {
          return throwError(null);
        }
      }),
      switchMap((user: any) => {
        console.log(user, 'user sign in');
```

```

        this._userService.user = user;
        return of(user);
      })
    );
}

getCurrentUser(): Observable<any> {
  return this._authApiService.getCurrentUser().pipe(
    map((_user: any) => {
      _user.profile.functionalities = JSON.parse(
        _user.profile.functionalities
      );
    });
  );
}

```

```

        return _user;
    })
    );
}

// Servizio AuthApiService
signIn(username: string, password: string): Observable<any> {
    return this._httpClient.post<any>(`${environment.api.auth}/1fa/authenticate`,
    {username, password});
}

getCurrentUser(): Observable<any> {
    return this._httpClient.get<any>(`${environment.api.userV2}/current`);
}

```

Se l'utente ha inserito i dati nel modo corretto e viene trovato all'interno del database, il backend ci restituisce un token. Questo token viene poi salvato nel session storage, in modo che, se riapriremo il sito in un secondo momento, non sarà necessario eseguire di nuovo il login. Quando eseguiamo il logout, verrà chiamato un metodo all'interno di AuthService che si occuperà di rimuovere il token dal session storage.

```

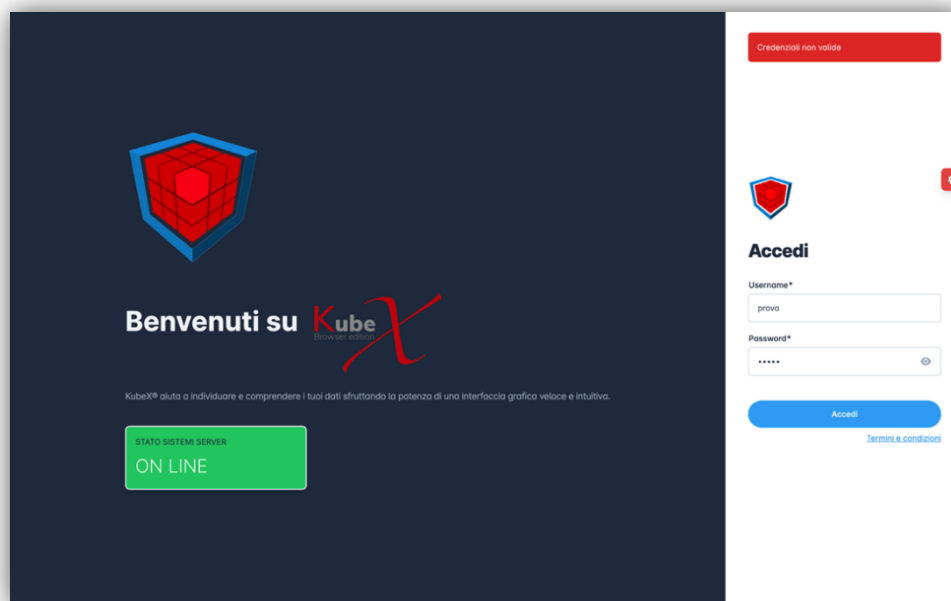
// Servizio AuthService
signOut(): Observable<any> {
    console.log('signout');
    // Remove the access token from the local storage
    sessionStorage.removeItem(ACCESS_TOKEN_STORAGE_KEY);

    // Set the authenticated flag to false
    this._authenticated = false;

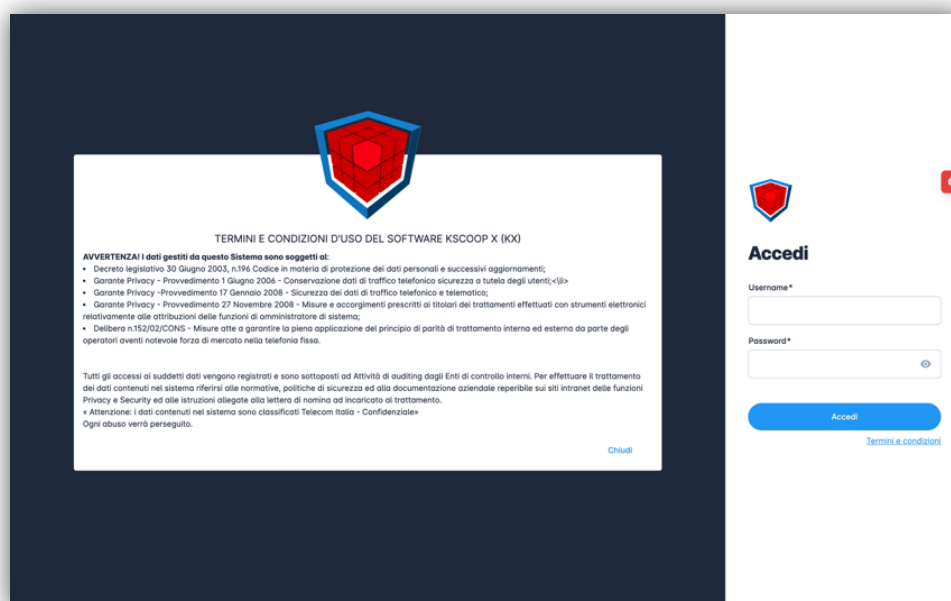
    // Return the observable
    return of(true);
}

```

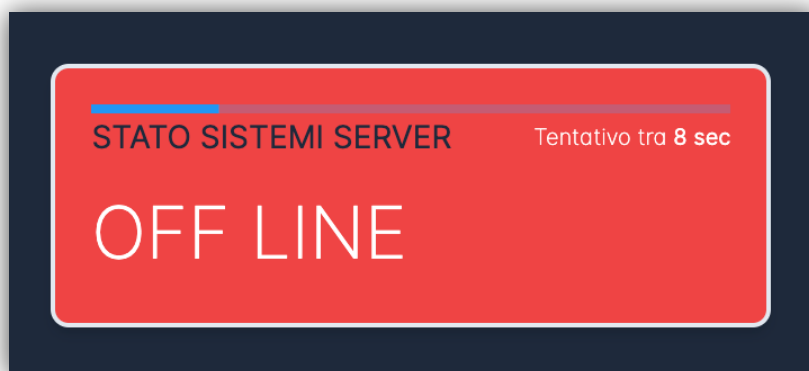
Nel caso in cui i dati inseriti non siano corretti, verrà visualizzato un alert al di sopra del form, così da informare l'utente dell'errore.



Il contenuto presente alla sinistra del form fa parte del componente Auth. Questo componente serve a mostrarci il testo di benvenuto e lo stato dei server, come mostrato sopra, ma se dovessimo cliccare sul testo “termini e condizioni”, posto al di sotto del form di login, ci mostrerà invece la sezione dedicata ai termini e condizioni per l’uso del software.



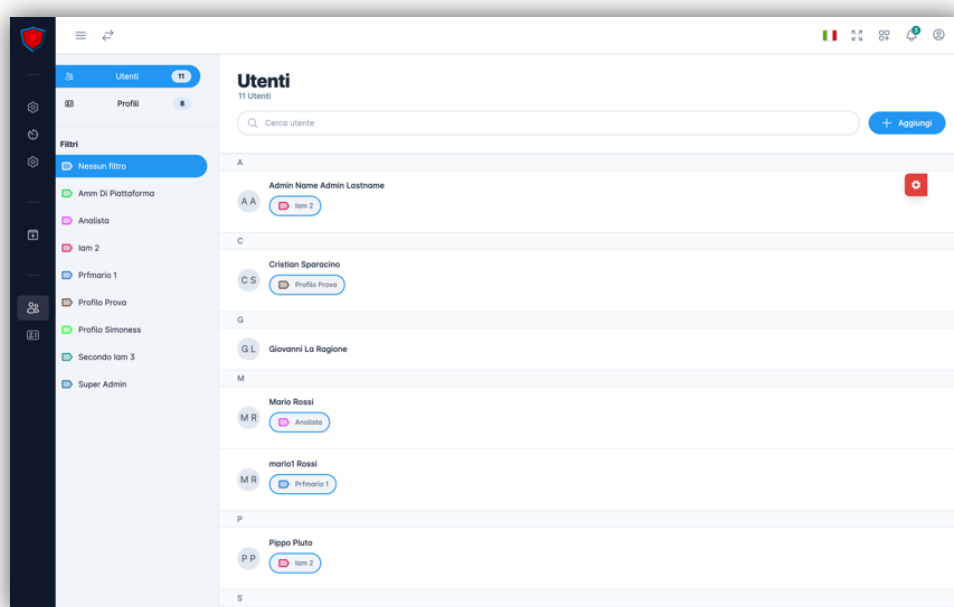
Il componente che ci permette di visualizzare lo stato dei server è il componente StatusServer. Se lo stato dei server è raggiungibile, lo visualizzeremo come nell’immagine soprastante, altrimenti in modo differente.



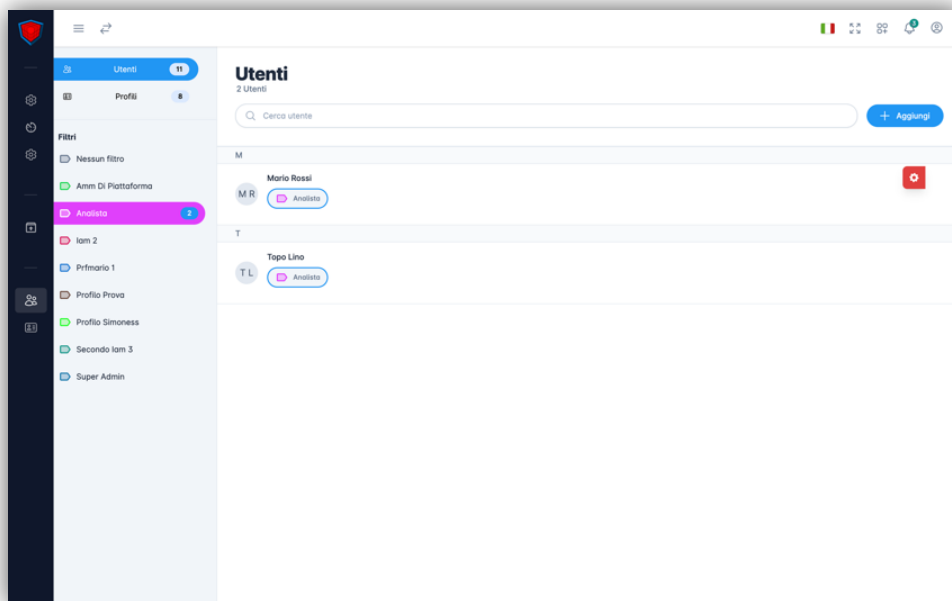
In caso di server non raggiungibile, verrà eseguito un nuovo tentativo di connessione dopo dieci secondi.

Utenti

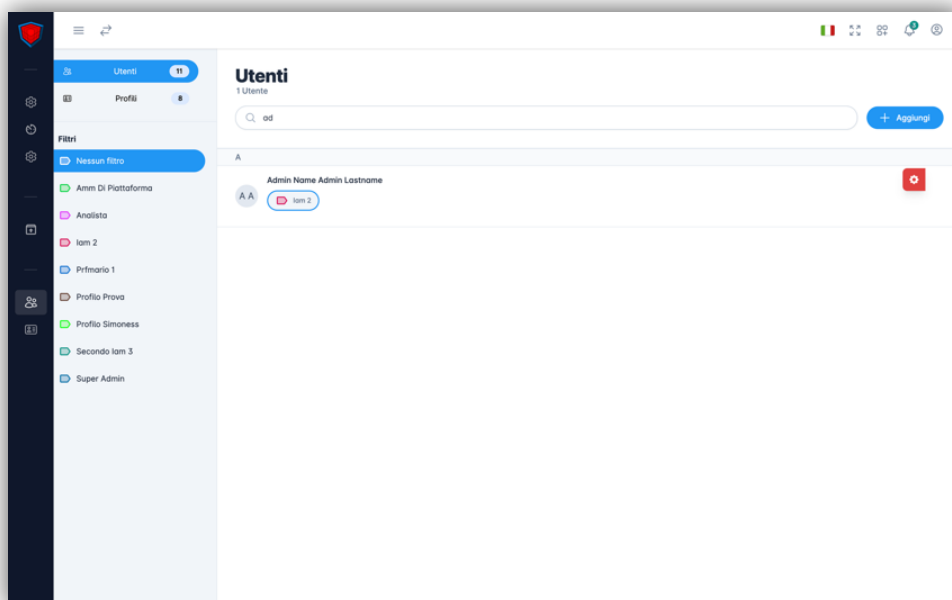
All'interno di questa pagina possiamo trovare tutti i componenti relativi all'utente.

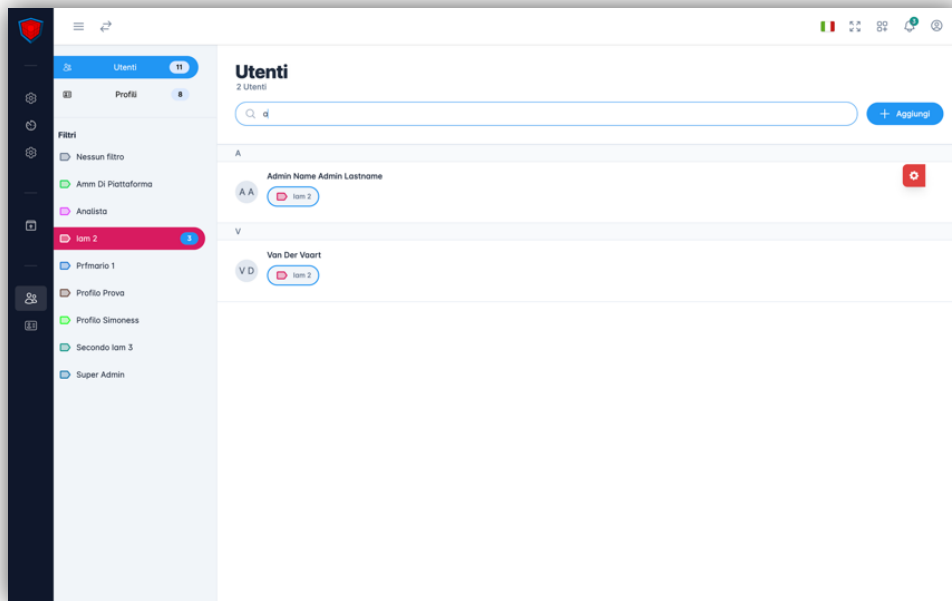


Come possiamo vedere, sul lato sinistro della pagina è presente una sidebar divisa in due sezioni. La prima sezione comprende un componente dedicato alla navigazione tra la pagina dei profili e quella degli utenti. Nella seconda parte troviamo la sezione dedicata ai filtri profilo. Ad ogni utente appartiene un solo profilo, quindi, selezionando il profilo da cercare, potremo vedere la lista utenti, posta nella parte centrale dello schermo, filtrata in base al profilo selezionato.



Una volta selezionato il profilo, oltre al filtraggio della lista utenti, noteremo che accanto al nome del profilo comparirà un numero che rappresenta quanti utenti sono associati a quel determinato profilo. La lista dei profili selezionabili è dinamica, in modo da aggiornarsi autonomamente nel caso in cui l'utente crei un nuovo profilo. Oltre alla ricerca per profilo associato, sarà possibile cercare anche per nome utente, oppure combinare entrambi i parametri.





Per rendere la navigazione più fluida, evitando situazioni di disagio in cui alcuni utenti vengono caricati mentre altri no, abbiamo deciso di inserire la chiamata per ottenere gli utenti all'interno della resolve della rotta profiles. In questo modo la pagina resterà in caricamento fino a quando non saranno caricate tutte le informazioni presenti nella resolve. L'utente, in questo modo, percepirà una navigazione più fluida, poiché aprendo un utente non dovrà aspettare il caricamento delle informazioni al momento, evitando quindi situazioni di caricamento asincrono visibile o comportamenti incoerenti nell'interfaccia, migliorando la percezione di velocità e continuità nell'esperienza utente.

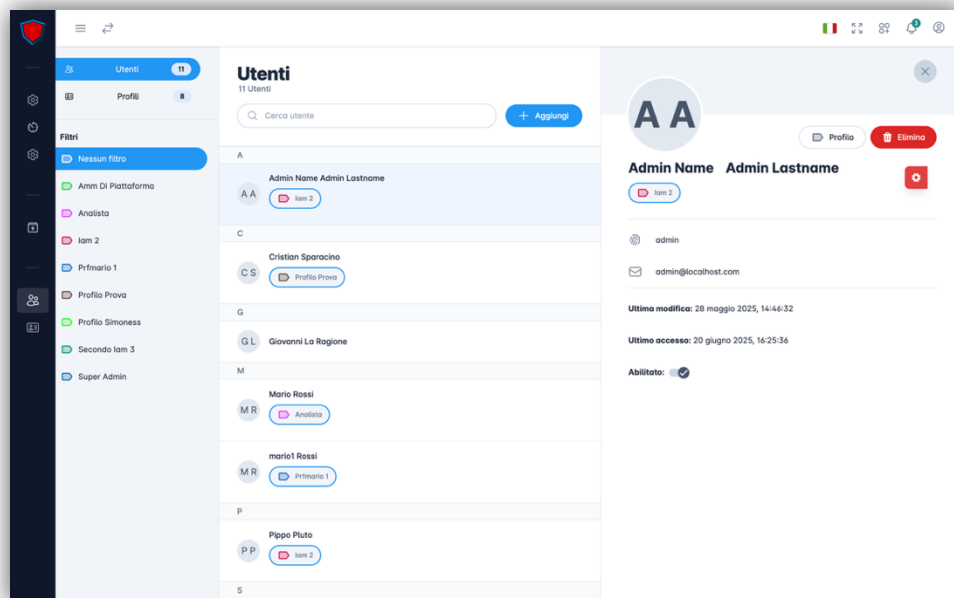
```
{
  path: '',
  component: AdminUsersComponent,
  resolve: {
    counters: async () => {
      const adminService: AdminService = inject(AdminService);
      adminService.counters = await lastValueFrom(
        inject(EntityApiService).getEntityCounter$()
      );
    },
    profiles: () => inject(AdminApiService).getProfiles(),
  },
  children: [
    {
      path: '',
      component: AdminUsersListComponent,
      resolve: {
        users: () => inject(AdminApiService).getUsers(),
      },
      data: {
        role: Role.ROLE_ADMIN_MANAGE_USR_PERMS,
      },
      canActivate: [AuthGuard, RoleGuard],
      children: [
```

```

    {
      path: 'details/:id',
      component: AdminUserDetailComponent,
      resolve: {
        user: (route: ActivatedRouteSnapshot) =>
inject(AdminUsersService).getUser(route.params['id']),
        profiles: () => inject(AdminApiService).getProfiles(),
      },
      data: {
        role: Role.ROLE_ADMIN_MANAGE_USR_PERMS,
      },
      canActivate: [AuthGuard, RoleGuard],
    },
    {
      path: 'new',
      component: AdminUserFormComponent,
      data: {
        role: Role.ROLE_ADMIN_MANAGE_USR_PERMS,
      },
      canActivate: [AuthGuard, RoleGuard],
    },
  ],
},
],
}

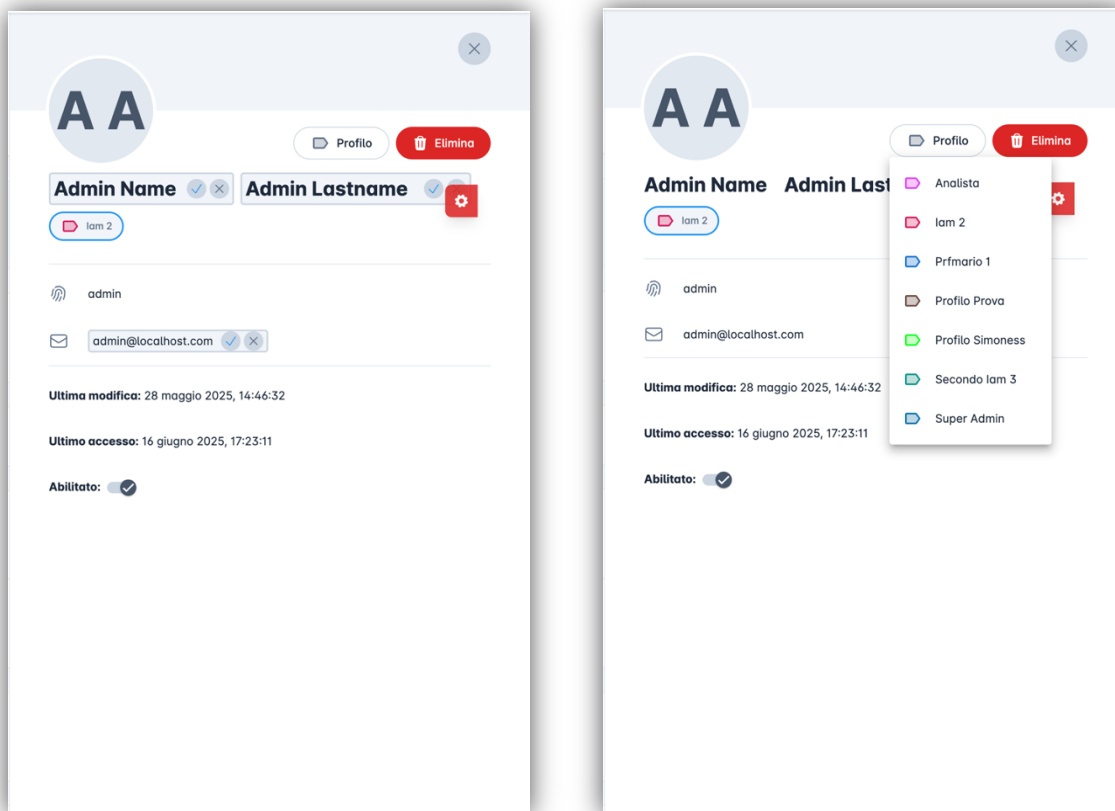
```

Come possiamo vedere, non tutte le rotte saranno accessibili a tutti gli utenti, ma solo a quelli che possiedono determinati ruoli. Selezionando un utente all'interno della lista verrà mostrato un drawer contenente le informazioni relative all'utente selezionato.



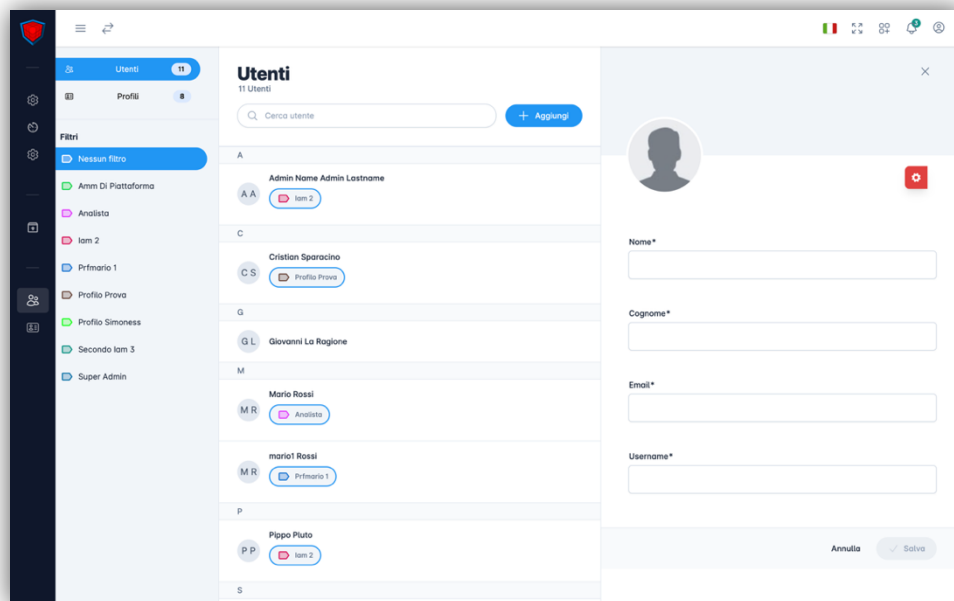
Queste informazioni vengono recuperate nella resolve della rotta details/:id, in modo da averle tutte a disposizione come indicato precedentemente. I dati modificabili si trovano all'interno di un

componente KxEditableLabel, che consente di visualizzare e modificare i valori, comunicando al backend le modifiche effettuate, così da poterle salvare. Per modificare un campo, sarà sufficiente cliccarci sopra; comparirà un bordo attorno al campo, contenente due bottoni: uno per confermare le modifiche e l'altro per annullarle e ripristinare il valore originale. Per modificare il profilo associato all'utente, occorrerà selezionare il campo "Profilo" e scegliere il profilo da associare.



Per eliminare un utente sarà sufficiente cliccare il bottone "Elimina", il quale aprirà un dialog di conferma. Se l'utente confermerà la decisione di eliminare l'utente selezionato, attraverso una chiamata DELETE all'API dedicata, l'utente verrà eliminato e rimosso dalla lista utenti. Nel caso in cui l'utente che si tenta di eliminare sia lo stesso con cui si è effettuato l'accesso, verrà mostrato un alert che informerà l'utente che non è possibile procedere poiché l'utente è attualmente in uso.

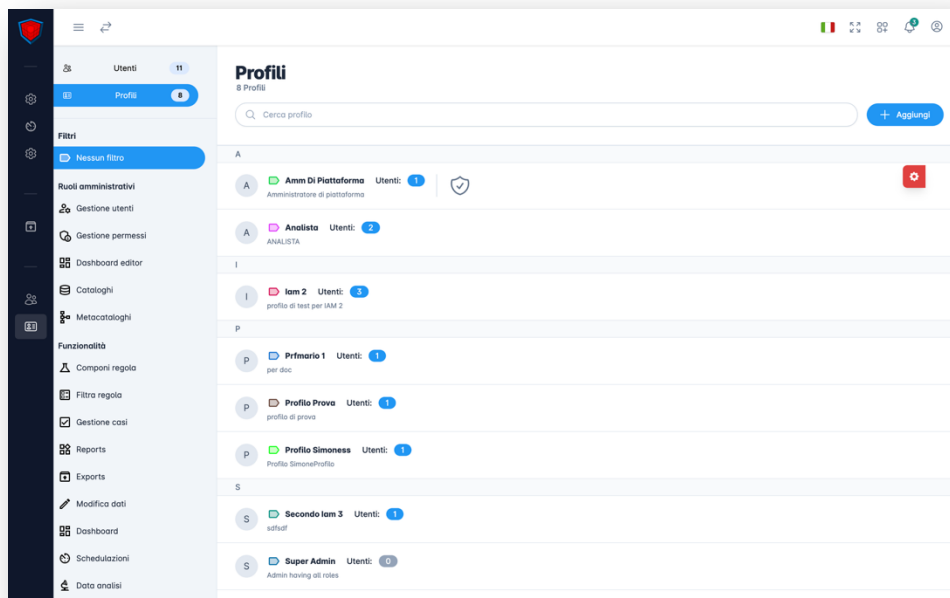
Per creare un nuovo utente, occorrerà selezionare il bottone "Aggiungi", posto nella parte superiore dello schermo. Una volta cliccato, si aprirà un drawer contenente un form per la creazione di un nuovo utente.



Dopo aver inserito i dati, sarà sufficiente cliccare sul pulsante “Salva” (in basso a destra). A questo punto, il front-end invierà le informazioni al back-end tramite una chiamata POST all’API dedicata. Se i dati saranno corretti, verrà mostrato all’utente un alert con un messaggio di creazione riuscita; in caso contrario, verrà mostrato un alert con un messaggio di errore.

Profili

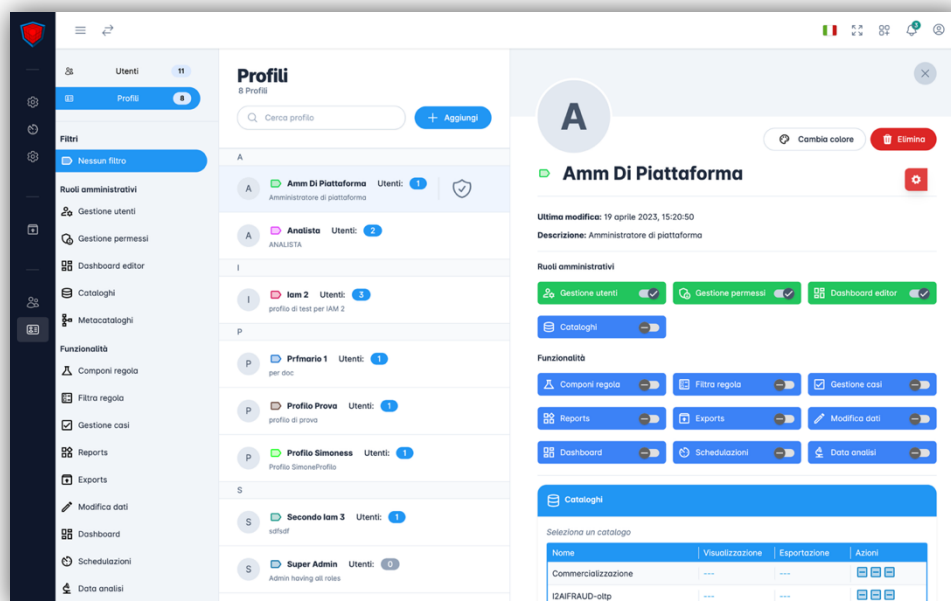
La pagina Profili, come quella dedicata agli utenti, è una pagina che ci mostra la lista dei profili creati. Anche qui la pagina è divisa in due parti: la parte sinistra, composta da una sidebar dedicata alla navigazione tra utenti e profili e ai filtri in base alle funzionalità e ai ruoli amministrativi; mentre a destra troviamo la lista dei profili.



Ogni profilo mostra il colore collegato ad esso, il nome, il numero di utenti collegati, la descrizione e se il profilo è un amministratore di sistema. I filtri e la lista dei profili, come per la pagina dedicata agli utenti, sono caricati dinamicamente all'interno della resolve della rotta del componente, tramite chiamate GET alle API dedicate nel back-end. Cliccando su un profilo, ci verrà mostrato il suo dettaglio; a differenza del dettaglio utente, il dettaglio profilo mostra più opzioni, tutte modificabili. Oltre alle informazioni principali relative all'account (nome, colore, ultima modifica e descrizione), troviamo tre sezioni.

La prima sezione, intitolata "Ruoli amministrativi", è dedicata all'assegnazione dei ruoli che il profilo possiede. Se il toggle è attivo, lo sfondo diventerà verde; mentre se disattivato, rimarrà blu, in modo da poter essere distinto da quello utente.

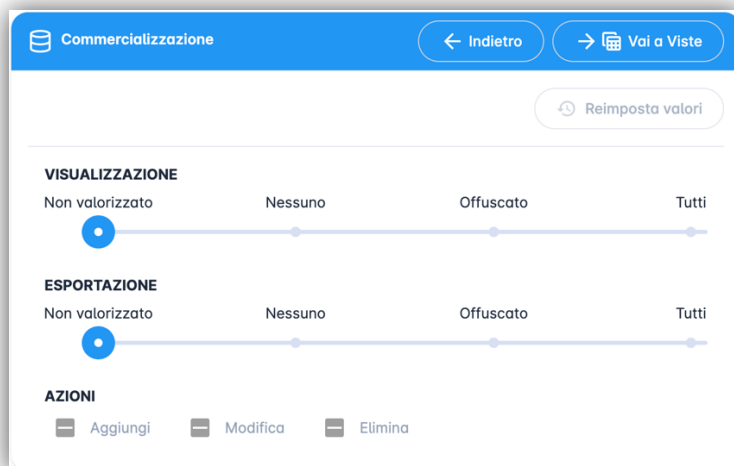
Nella seconda sezione troviamo invece la lista delle funzionalità a cui il profilo può accedere; se una funzionalità sarà disattivata, l'utente non vedrà la pagina dedicata alla funzionalità all'interno della sidebar di navigazione. Se dovesse provare ad accedere alla pagina tramite URL, con la funzionalità disattivata, verrà reindirizzato a una pagina che lo informa dell'impossibilità di accedere per via delle impostazioni del profilo.



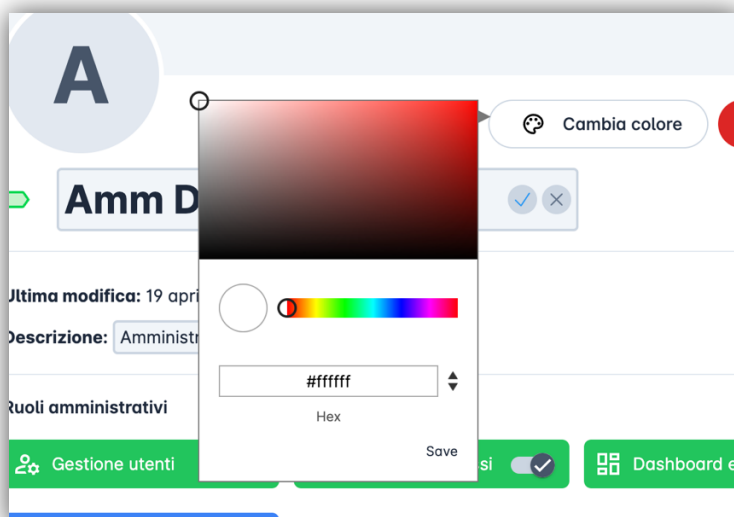
Al di sotto delle opzioni di funzionalità troviamo una sezione dedicata ai cataloghi collegati al profilo e alle azioni che sarà possibile eseguire con essi. In questa sezione possiamo decidere quali azioni (aggiunta, modifica o eliminazione) il profilo potrà eseguire sul catalogo selezionato o sugli elementi che lo compongono.

Cataloghi			
Seleziona un catalogo			
Nome	Visualizzazione	Esportazione	Azioni
Commercializzazione	---	---	  
I2AIFRAUD-oltp	---	---	  
I2ALOGGER-oltp	---	---	  
KX-query-oltp	---	---	  
New Int	---	---	  

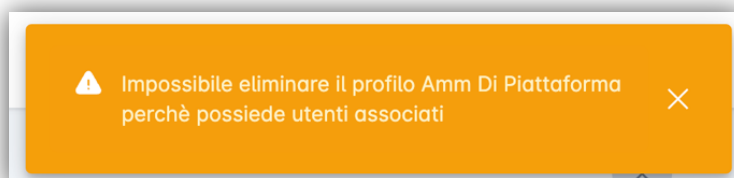
Ci basterà selezionare un catalogo per essere portati nella sezione dedicata alla modifica delle possibili interazioni con esso. Cliccando il bottone in alto a destra, potremo continuare ad addentrarci all'interno degli elementi che compongono il catalogo, per modificare le interazioni con i singoli elementi.



Per la selezione del colore associato al profilo ci siamo affidati alla libreria ngx-color-picker.

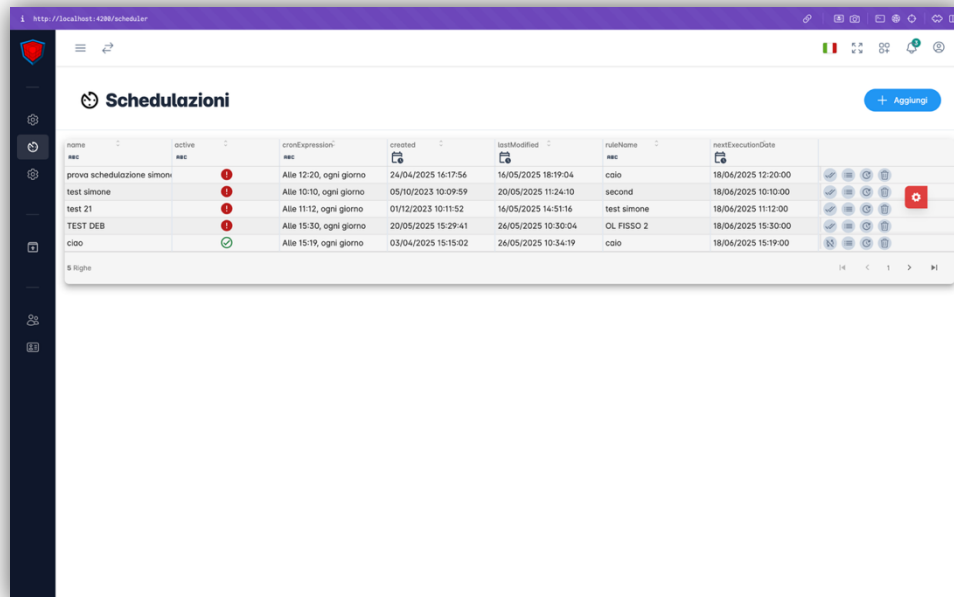


Per quanto riguarda la creazione e l'eliminazione di un profilo, i passaggi sono gli stessi della pagina utenti, con la differenza che non sarà possibile eliminare un profilo associato a uno o più utenti. In caso di errori, verremo notificati attraverso alert.



Le schedulazioni

All'interno della pagina Schedulazioni possiamo trovare la lista delle schedulazioni. La lista ci viene mostrata tramite il componente data-table.



name	active	cronExpression	created	lastModified	ruleName	nextExecutionDate
prova schedulazione simone		Alte 12:20, ogni giorno	24/04/2025 16:17:56	16/05/2025 18:19:04	colo	18/04/2025 12:20:00
test simone		Alte 10:10, ogni giorno	05/10/2023 10:09:59	20/05/2025 11:24:10	second	18/04/2025 10:10:00
test 21		Alte 11:12, ogni giorno	01/12/2023 10:11:52	16/05/2025 14:51:16	test simone	18/04/2025 11:12:00
TEST DEB		Alte 15:30, ogni giorno	20/05/2025 15:29:41	26/05/2025 10:30:04	OL FISSO 2	18/04/2025 15:30:00
ciao		Alte 15:19, ogni giorno	03/04/2025 15:15:02	26/05/2025 10:34:19	colo	18/04/2025 15:19:00

Ogni schedulazione viene recuperata dal back-end tramite una chiamata GET all'API dedicata. Come spiegato in precedenza, per poter utilizzare il componente data-table, sarà necessario convertire i dati da inserire nella tabella. Per convertirli, utilizzeremo la seguente callback.

```
apiCallback = (): Observable<any> => {
  return this._schedulerService.getAllScheduleFile().pipe(
    map((data: IDataTableData) => {
      const columnsNameShown: string[] = ['name', 'active', 'created',
      'lastModified', 'ruleName', 'nextExecutionDate', 'cronExpression'];
      let columnType: Type;
      let columnStatusProp: any;
      let columnCronoProp: any;
      let rowActions: IDataTableAction[] = [];
      return {
        ...data,
        columns: data.columns.map((_column: IDataTableColumn):
IDataTableColumn => {
          if (_column.name === 'created' || _column.name ===
'lastModified' || _column.name === 'nextExecutionDate') {
            columnType = Type.DateTime;
          } else {
            columnType = Type.String;
          }

          if (_column.name === 'active') {
            columnStatusProp = _column.prop;
          }
        })
      }
    })
  );
}
```

```

        if (_column.name === 'cronExpression') {
            columnCronoProp = _column.prop;
        }

        return {
            ..._column,
            type: columnType,
            hidden: !columnsNameShown.includes(_column.name),
            sortable: !(_column.name === 'status'),
        };
    })),
    rows: data.rows.map((_row: IDDataTableRow) => {
        if (_row.rowItem[columnStatusProp]) {
            rowActions = [
                {
                    id: ScheduleDataTableRowActions.DISABILITA,
                    icon: 'mat_outline:sync_disabled',
                    tooltip: 'DISABILITA',
                },
                ...this.dataTableRowActions,
            ];
        } else {
            rowActions = [
                {
                    id: ScheduleDataTableRowActions.ABILITA,
                    icon: 'mat_outline:done_all',
                    tooltip: 'ABILITA',
                },
                ...this.dataTableRowActions,
            ];
        }
    })

    // Sostituisce il valore della colonna Status con l'icona associata
    switch (_row.rowItem[columnStatusProp]) {
        case true:
            _row.rowItem[columnStatusProp] = {
                icon: 'mat_outline:check_circle',
                cssClass: 'text-green-700',
                originalValue: _row.rowItem[columnStatusProp],
            } as IDDataTableIconValue;
            break;
        case false:
            _row.rowItem[columnStatusProp] = {
                icon: 'mat_solid:error',
                cssClass: 'text-red-700',
                originalValue: _row.rowItem[columnStatusProp],
            } as IDDataTableIconValue;
            break;
    }

    _row.rowItem[columnCronoProp] =
this.cronToTextPipe.transform(_row.rowItem[columnCronoProp] as string);
    _row.rowActions = rowActions;
    return _row;
})),
});
});
);

```

```
};
```

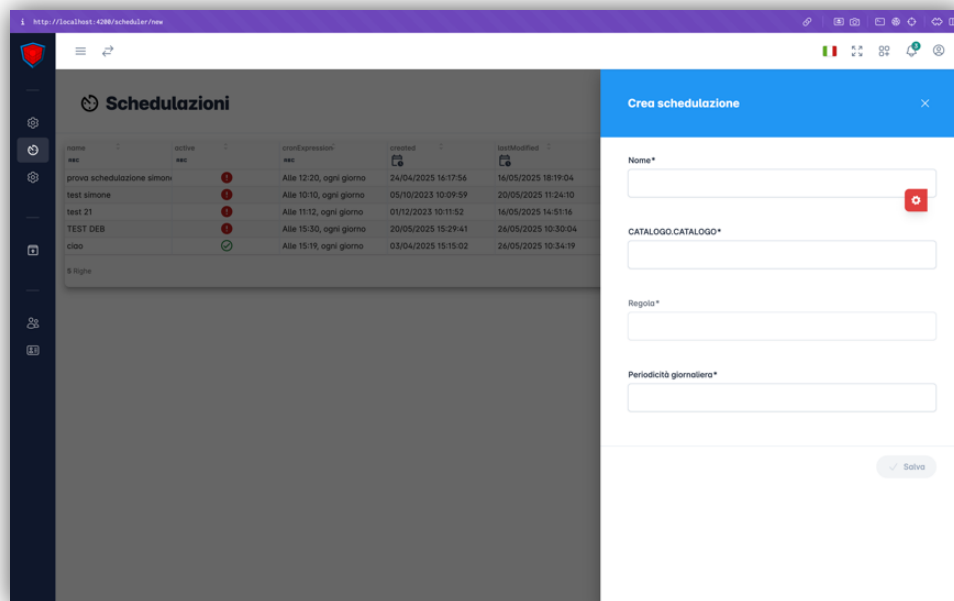
Osservando il codice, possiamo notare che all’inizio della callback andremo a definire i nomi delle colonne, e successivamente assegneremo i valori in base alla riga in cui ci troveremo. Possiamo osservare come, nella parte finale della callback, venga utilizzato il metodo transform associato alla pipe CronToText.

```
transform(value: string, useLocalTime: boolean = true): string {  
    const appLocale: string = this._translocoService.getActiveLang()  
    const cronValue = (useLocalTime) ?  
    Utils.convertServerCronToLocalTimeCron(value) : value;  
    return crontrue.toString(cronValue, {verbose: true, use24HourTimeFormat:  
true, locale: appLocale});  
}
```

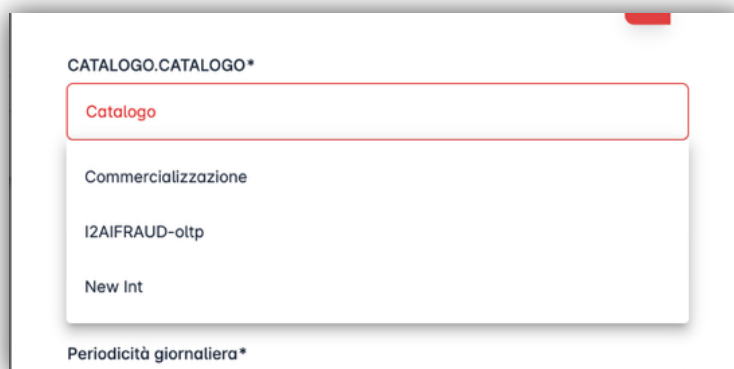
Il metodo transform, associato alla pipe CronToText, viene utilizzato per convertire il valore di una variabile contenente un’espressione cron (es. 0 20 10 * * ? *) in una stringa più intuitiva per l’utente (es. "Alle 12:20, ogni giorno"), tramite un metodo contenuto nelle Utils del progetto.

```
public static convertServerCronToLocalTimeCron(cronExpression: string): string {  
    const cronSplitted = cronExpression.split(' ');  
    const cronExpressionHour = cronSplitted[2];  
    const cronExpressionMinute = cronSplitted[1];  
    const timezoneArea = moment.tz.guess();  
    const tz = moment().tz(timezoneArea).format('Z');  
    const tzSplitted = tz.split(':');  
    const tzHour = tzSplitted[0];  
    const operator = tzHour[0];  
    const tzHourFormatted = tzHour[1] === '0' ? tzHour[2] : tzHour[0];  
    let hourWithTz;  
    if (operator === '+') {  
        hourWithTz = Number(cronExpressionHour) + Number(tzHourFormatted);  
    } else {  
        hourWithTz = Number(cronExpressionHour) - Number(tzHourFormatted);  
    }  
    return this.getCronExpression(cronExpressionMinute, hourWithTz);  
}  
  
public static getCronExpression(minute: string, hour: string): string {  
    return `0 ${minute} ${hour} * * ? *`;  
}
```

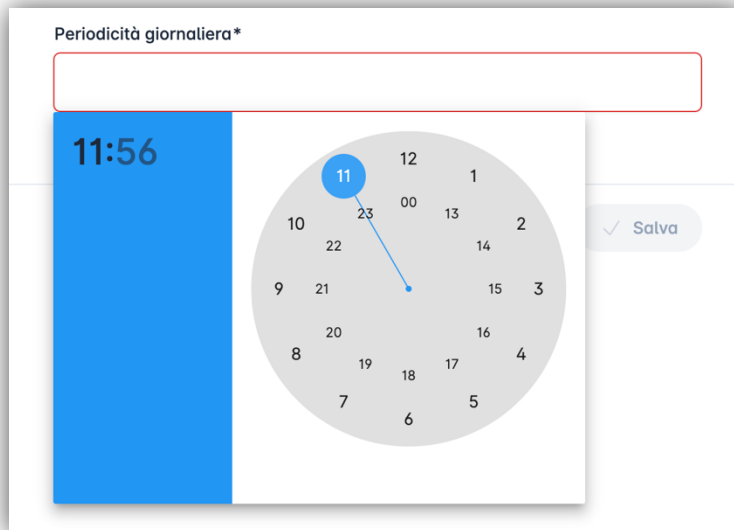
Per poter creare una schedulazione, occorrerà selezionare il bottone posto in alto a destra; una volta cliccato, si aprirà un drawer contenente il form per la creazione di una schedulazione.



La selezione della regola non sarà disponibile fino a quando non sarà stato selezionato un catalogo, in quanto ogni regola è assegnata a un solo catalogo.



Per selezionare l'orario in cui eseguire la schedulazione, ci siamo affidati alla libreria MatDatePicker.



Una volta selezionato il valore all'interno del form, esso verrà visualizzato tramite la pipeline citata in precedenza. Ogni schedulazione avrà quattro interazioni possibili, poste alla fine della riga nella tabella:

Eliminazione

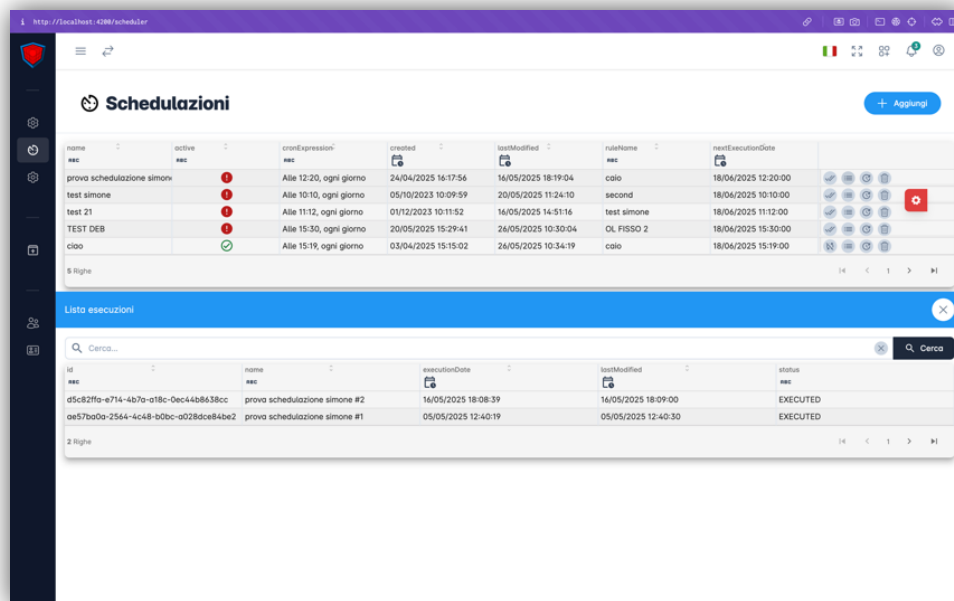
La schedulazione potrà essere eliminata tramite una chiamata DELETE all'API dedicata.

Cambia stato

Ogni schedulazione potrà cambiare stato da attivata a disattivata e viceversa. Per migliorare l'esperienza utente, abbiamo deciso di modificare dinamicamente le icone in base allo stato della schedulazione.

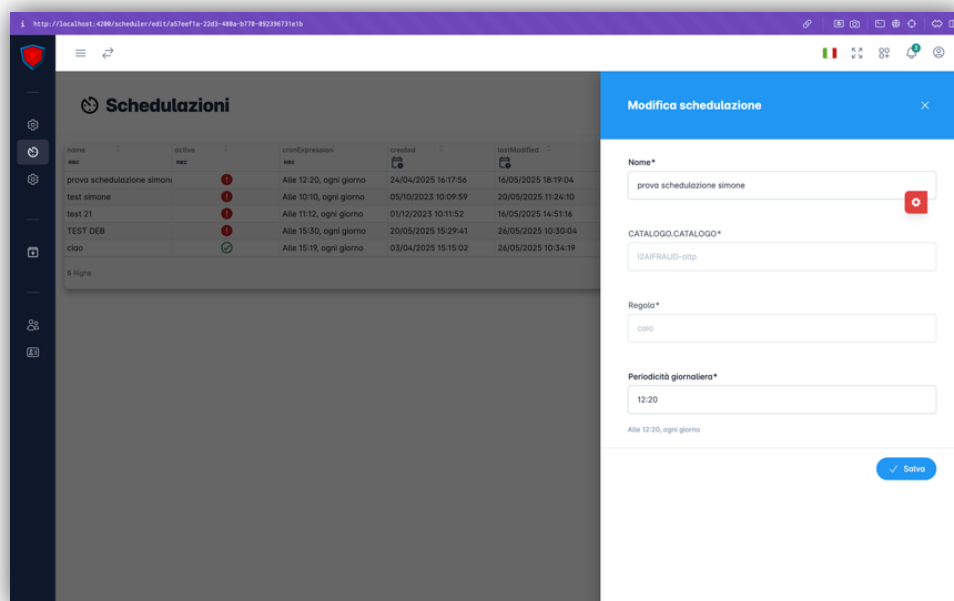
Lista esecuzioni

Selezionando il pulsante dedicato, sarà possibile visualizzare la lista delle esecuzioni della schedulazione, con le relative informazioni. Queste informazioni sono contenute all'interno di un componente custom chiamato data-table-bottom-bar: un componente che permette di mostrare una data table a comparsa dalla parte inferiore dello schermo. Per ottenere la lista delle esecuzioni sarà eseguita una chiamata GET all'API del back-end.



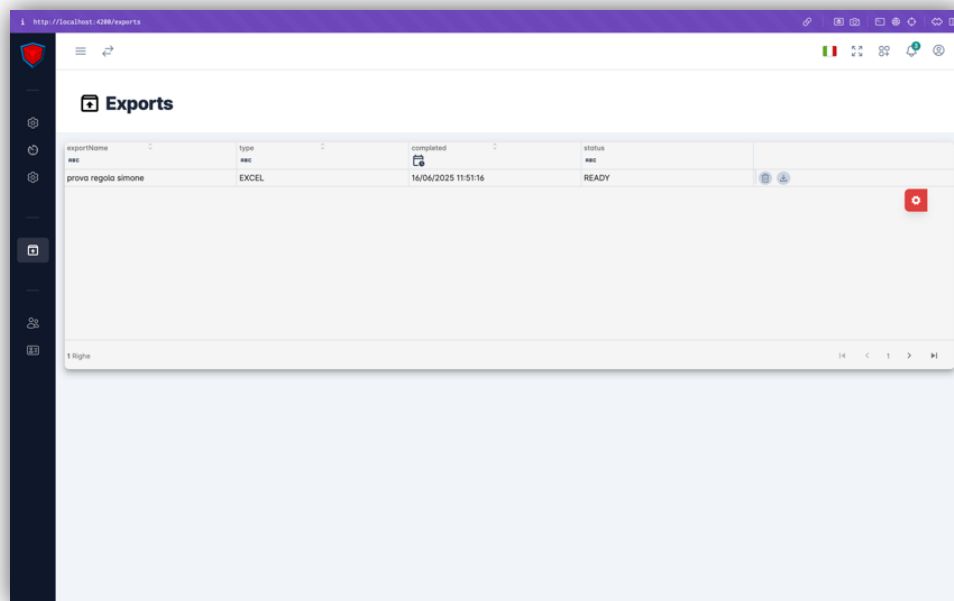
Modifica schedulazione

Sarà possibile modificare i dati relativi all'esecuzione della schedulazione, come il nome e il momento della giornata in cui andrà eseguita.



Export

All'interno della pagina dedicata agli export, potremo visualizzare la lista degli export creati all'interno di una data-table.



Le interazioni possibili con ogni riga saranno:

Download

Sarà possibile scaricare il file excel o csv con le informazioni salvate. Per poter scaricare il file utilizzeremo la seguente logica:

```
actionDownloadFile(action: IDataTableActionOutput): void {
  const rowItem: IDataTableRowItem = action.data.rowItem;
  this._exportService.doDownloadFile(action).catch(() => {
    this.dataTableComponent.reloadData();
    this._snackBarService.error({
      message:
this._translocoService.translate('MESSAGE.EXPORT_SCARICAMENTO_FALLITO', {
      nome: rowItem?.exportName,
    })),
  });
});
}

doDownloadFile(file: IDataTableActionOutput): Promise<void> {
  const row: IDataTableRowItem = file.data.rowItem;
  const _fileName = `${row.exportName}.${ExportFileExtantions[(row as
any).type]}`;

  return new Promise((resolve, reject) => {
    this._exportApiService.downloadExport$(row.id).subscribe({
      next: (response) => {
        try {
          Utils.downloadFile(response, _fileName, row.type as string);
          resolve();
        } catch (error) {
          reject();
        }
      }
    },
  ),
}
```



```

        error: reject,
    });
});
}

downloadExport$(fileId: any): Observable<any> {
    return this._httpClient.get(`${environment.api.export}/${fileId}/download`, {
        responseType: 'arraybuffer', observe: 'response' as 'body' });
}

public static downloadFile(response, fileName: string, fileType: string): void {
    if (response?.body) {
        try {
            const fileNameTemp = !!response.headers.get('Content-Disposition')
                ? response.headers.get('Content-Disposition').substring(21,
response.headers.get('Content-Disposition').length)
                : fileName;
            const fileTypeTemp = !!fileNameTemp.split('.').pop() ?
fileNameTemp.split('.').pop() : ExportFileExtantions[fileType];

            const blob: any = new Blob([response.body], { type: fileTypeTemp });
            const docElement = document.createElement('a');

            docElement.href = URL.createObjectURL(blob);
            docElement.download = fileNameTemp;
            docElement.click();
        } catch (error) {
            throw new Error(error);
        }
    } else {
        throw new Error();
    }
}
}

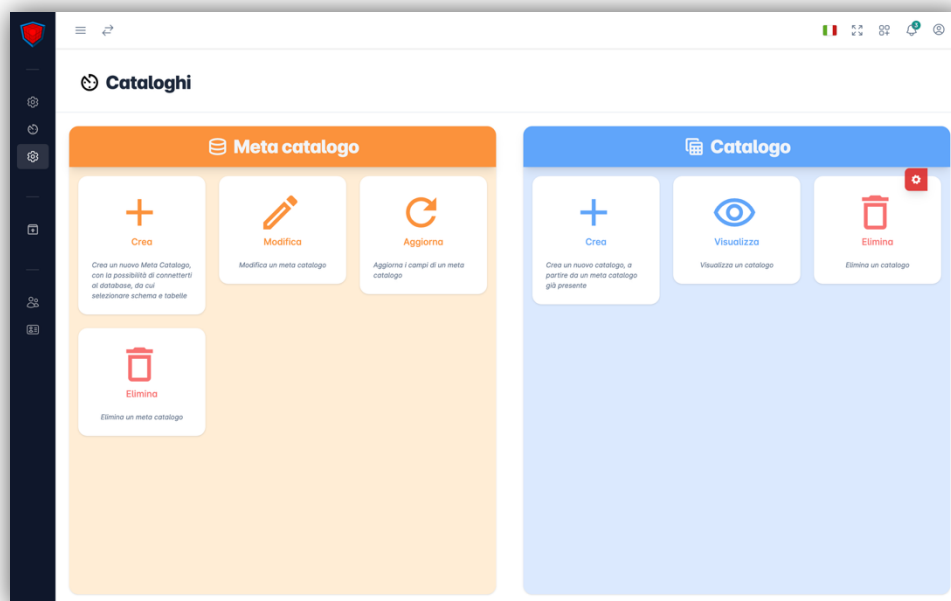
```

Eliminazione

Sarà possibile eliminare l'export attraverso una chiamata DELETE all'API dedicata.

I cataloghi

L'ultima pagina che possiamo trovare è quella dedicata a cataloghi e meta cataloghi.



Una volta entrati nella pagina, ci troveremo davanti a una schermata divisa in due sezioni: a sinistra quella dedicata ai meta cataloghi e a destra quella dedicata ai cataloghi.

Meta cataloghi

Nel lato sinistro della pagina troveremo quattro card contenenti le interazioni disponibili con i meta cataloghi.

Creare un meta catalogo

Una volta selezionata la card dedicata, verremo reindirizzati su una pagina con un drawer sempre aperto. All'interno della sidebar del drawer avremo la possibilità di muoverci tra:

- il componente dedicato alla connessione con un database esterno
- il componente dedicato alla creazione del meta catalogo

Il contenuto del drawer (ovvero il lato destro della pagina) mostrerà dinamicamente il componente selezionato. Il componente Schemas, nella sidebar, resterà disabilitato finché non sarà stabilita una connessione con un database; non sarà quindi possibile accedervi senza prima completare la connessione. Il passaggio tra i due componenti non avviene tramite rotte, ma in modo dinamico nel contenuto del drawer padre.

Per gestire i form di connessione e creazione, abbiamo deciso di utilizzare un form unico creato con la classe FormBuilder di Angular, suddiviso in due gruppi:

- `olapConnection`, per i dati relativi alla connessione al database esterno
- `schemas`, per i dati della creazione del meta catalogo

```

this.metacatalogForm = this._formBuilder.group({
  olapConnection: this._formBuilder.group({
    olapDriver: [null],
    jdbcUrl: [null],
    username: [null],
    password: [null],
    dataGenSchema: [null],
  }),
  schemas: this._formBuilder.group({
    name: [null],
    description: [null],
    tz: [null],
    schemas: [[]],
  }),
});

```

Per inserire i dati nel form padre a partire da un componente figlio, utilizziamo un input che prende il nome del gruppo predefinito (olapConnection). Nel componente figlio creiamo un nuovo FormGroup e gli assegniamo come reference il gruppo del form nel componente padre, in modo da poter interagire con esso direttamente.

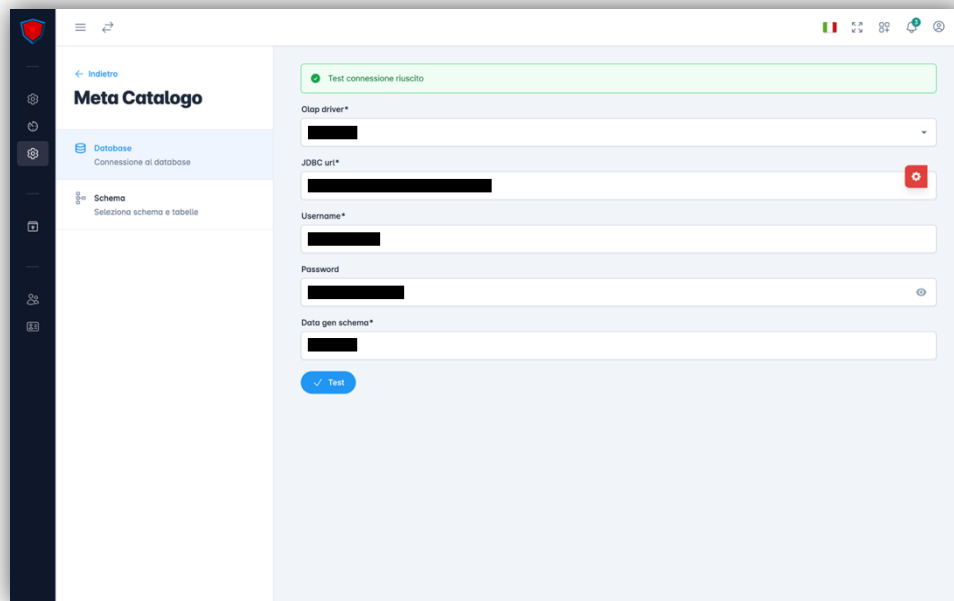
```

@Input() formGroupName: string;
form: FormGroup;

ngOnInit(): void {
  this.form = this.rootFormGroup.control.get(this.formGroupName) as FormGroup;
}

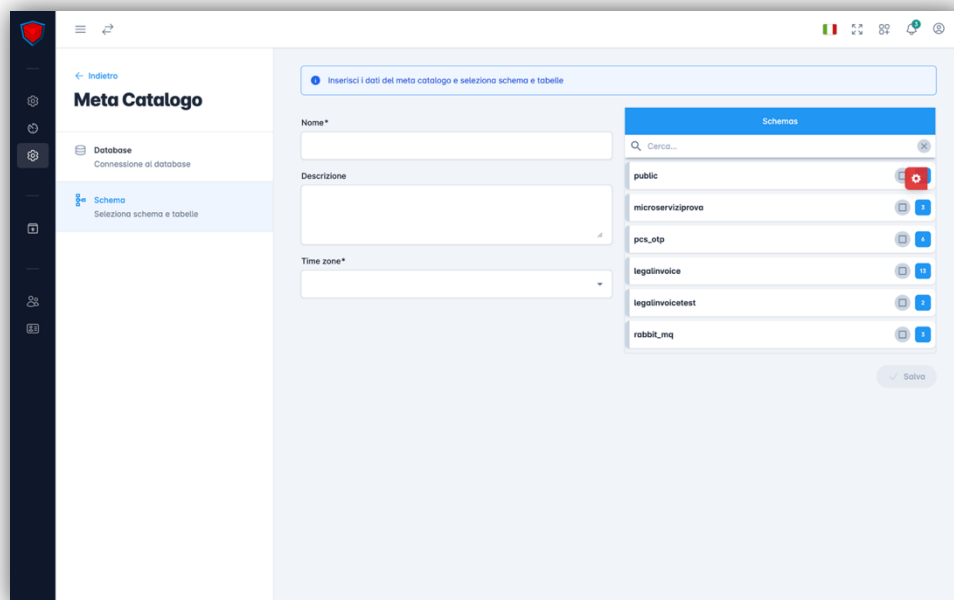
```

Se i dati inseriti saranno corretti e la connessione al database avrà successo, un alert in cima al form ci informerà, e il componente Schemas nella sidebar verrà abilitato. Se invece i dati saranno errati, un alert di errore ci avviserà.



The screenshot shows the 'Meta Catalogo' interface. On the left sidebar, the 'Schema' section is selected. The main area contains a form for database connection details. At the top, a green alert bar indicates 'Test connessione riuscito'. The form fields are: 'Olap driver*' (dropdown), 'JDBC url*' (text input with a red error icon), 'Username*' (text input), 'Password' (password input with an eye icon), and 'Data gen schema*' (text input). A blue 'Test' button is at the bottom of the form.

Una volta selezionato Schemas, il drawer mostrerà il relativo contenuto: un form per inserire nome, descrizione e time zone, e il componente custom AccordionNavigator.



The screenshot shows the 'Meta Catalogo' interface with the 'Schema' section selected in the sidebar. The main area has a blue header bar with the text 'Inserisci i dati del meta catalogo e selezione schema e tabelle'. Below this, there are three form fields: 'Nome*' (text input), 'Descrizione' (text input), and 'Time zone*' (dropdown). To the right of these fields is a custom 'Schemas' component, which is an AccordionNavigator. It contains a list of schemas: 'public', 'microservizi prova', 'pci_otp', 'legalinvoice', 'legalinvoicetest', and 'rabbitmq'. Each schema has a checkbox and a small icon. A 'Salva' button is at the bottom right of the form.

All'interno dell'AccordionNavigator troveremo la lista di schemas e tabelle (elementi nel database con cui abbiamo stabilito la connessione) da associare al meta catalogo. Il componente sarà diverso rispetto a quanto descritto nel capitolo precedente: qui sarà adattato per mostrare gruppi di checkbox.

Un'altra differenza rilevante è nella struttura: mentre prima era composta da category, folder e item, in questo caso visualizzeremo solo folder e item, senza la category.

La selezione delle checkbox avverrà dinamicamente tramite metodi pensati per gestire i diversi stati:

- tutte le caselle selezionate
- solo alcune selezionate
- nessuna selezionata

Dopo il controllo, passeremo all'AccordionNavigator l'elemento (item o folder) per cui aggiornare le icone. Se viene selezionata una folder, verrà chiamato un metodo specifico per gestirla.

```
updateAccordionNavigatorItem(event: IAccordionNavigatorItemOutput, icon:
IAccordionIcon[], isSelected: boolean) {
  this.accordionNavigatorSchema.updateItemFromExternal({
    folderIndex: event.folderIndex,
    itemIndex: event.itemIndex,
    categoryIndex: event.categoryIndex,
    configIndex: event.accordionConfigIndex,
    icon: icon,
    isSelected: isSelected,
  });
}

this._updateItem$.subscribe({
  next: (data: IAccordionNavigatorChangeIcon) => {

    this.accordionConfigList[data.configIndex].categories[data.categoryIndex].folders[
data.folderIndex].items[data.itemIndex].isSelected = data.isSelected;

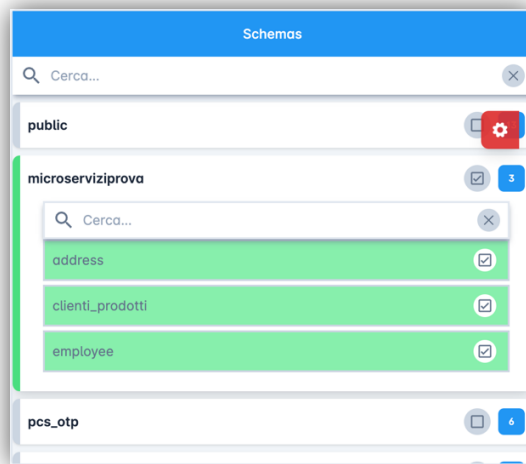
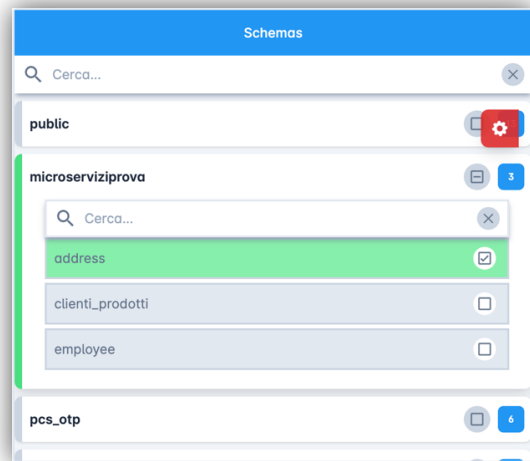
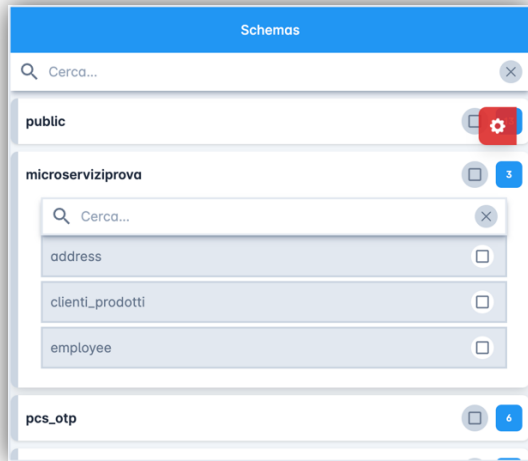
    this.accordionConfigList[data.configIndex].categories[data.categoryIndex].folders[
data.folderIndex].items[data.itemIndex].itemIcon = data.icon;
    this.accordionConfigList = cloneDeep(this.accordionConfigList);

    // Viene aggiornata anche la lista originale

    this.originalAccordionConfigList[data.configIndex].categories[data.categoryIndex].
folders[data.folderIndex].items[data.itemIndex].isSelected = data.isSelected;

    this.originalAccordionConfigList[data.configIndex].categories[data.categoryIndex].
folders[data.folderIndex].items[data.itemIndex].itemIcon = data.icon;
  },
});
```

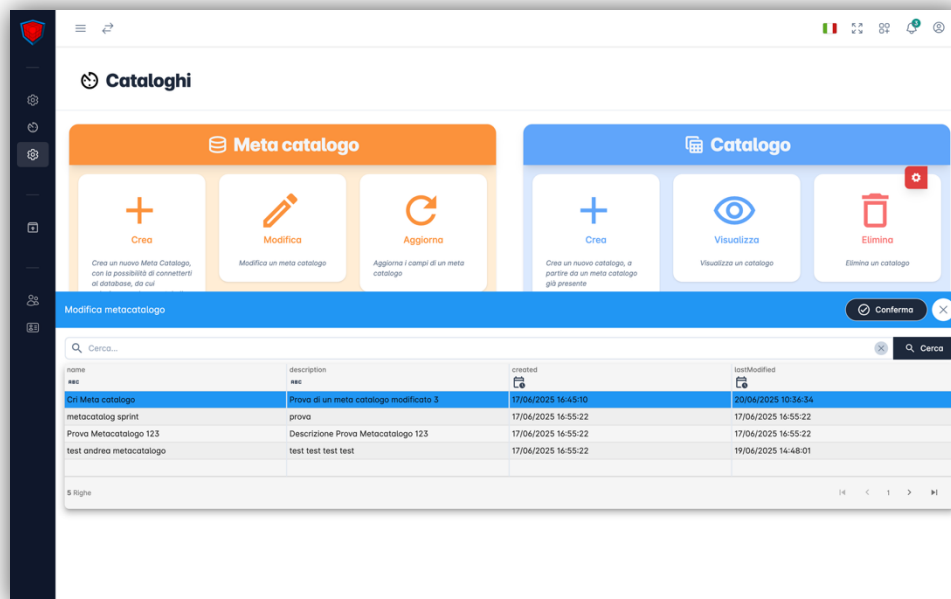
A seguire, le tre casistiche di visualizzazione dell'AccordionNavigator in base agli item selezionati.



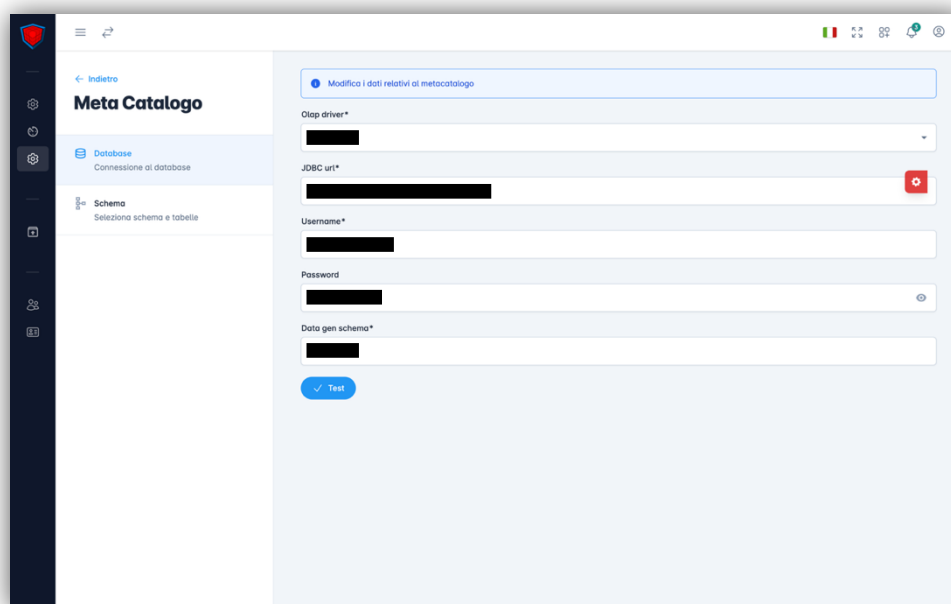
Una volta inseriti tutti i dati richiesti per la creazione, sarà possibile confermare il form: tramite una chiamata POST all'API dedicata nel back-end, verrà creato il meta catalogo. Un alert nella parte superiore del drawer ci informerà del successo o del fallimento dell'operazione.

Modificare un meta catalogo

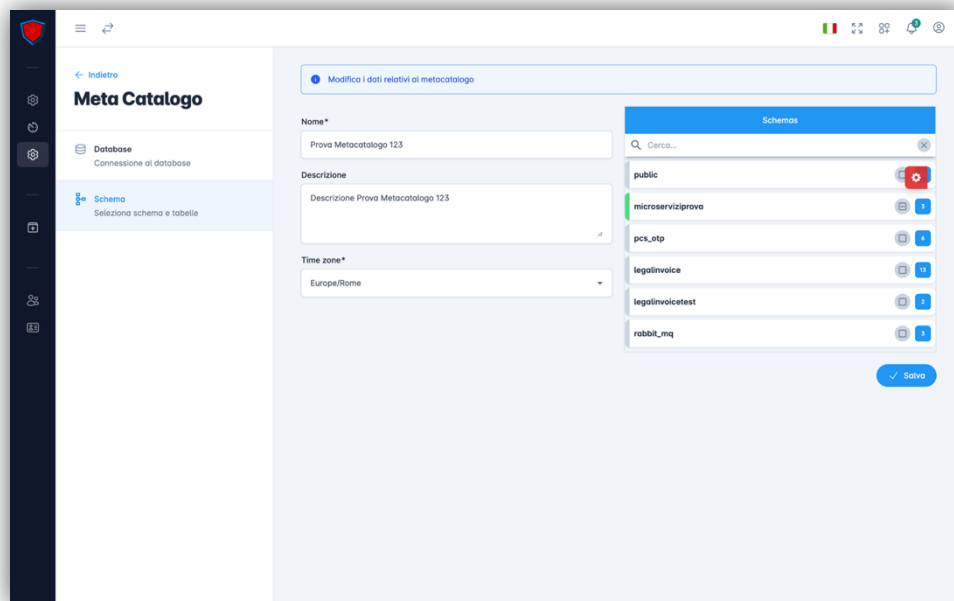
Una volta selezionata la card "Modifica", verrà aperta una data-table-bottom-bar contenente la lista dei meta cataloghi, dalla quale andremo a selezionarne uno da modificare.



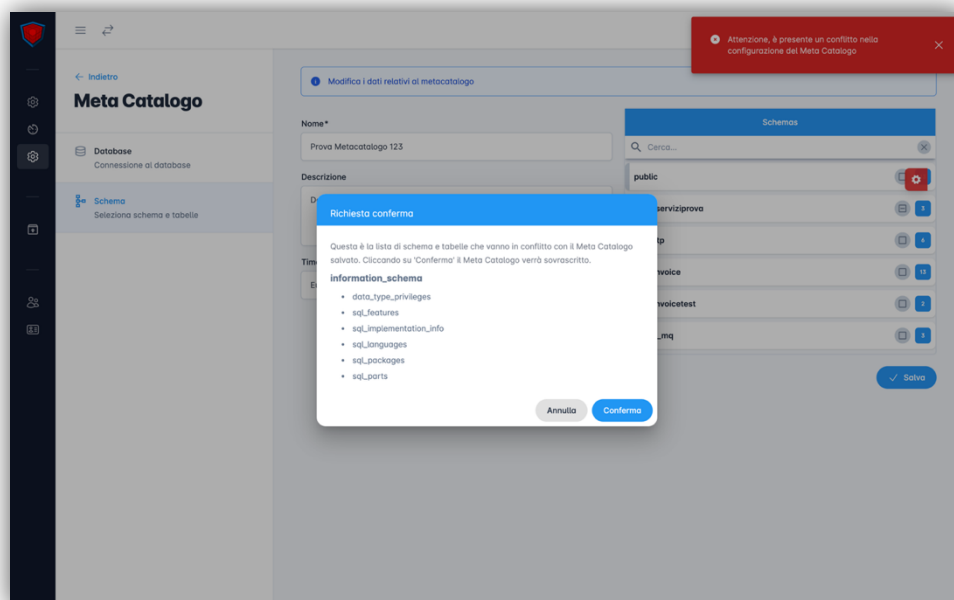
Dopo aver selezionato il meta catalogo da modificare, verremo portati nella pagina dedicata alla modifica, ovvero lo stesso componente della creazione, con la differenza che in questo caso passeremo un id all'interno della rotta, con il quale verrà eseguita una richiesta di tipo GET all'API nel backend.



Troveremo il form di connessione già compilato e la connessione già stabilita, in modo da non dover reinserire i dati ogni volta che si modifica un database; queste azioni avvengono all'interno dell'inizializzazione del componente, ma solo se nella rotta edit viene passato l'id di un meta catalogo esistente. Successivamente ci basterà cliccare sul componente Schemas per poter modificare gli elementi del meta catalogo, dove troveremo i form già popolati con gli elementi attuali del meta catalogo.



Per modificare gli elementi sarà sufficiente intervenire sui campi del form e salvare i dati tramite il bottone dedicato, che effettuerà una chiamata al backend per aggiornare i valori nel database. In caso di errori di conflitto (come tabelle o schema rimossi dal database) verremo avvisati tramite un dialog, attraverso il quale potremo forzare la modifica rimuovendo gli schema e le tabelle non più presenti.



Aggiornare un meta catalogo

Quando creiamo un meta catalogo, gli associamo degli schemas e delle tabelle provenienti da un database esterno. Se questi schemas o tabelle dovessero essere eliminati, il meta catalogo non

rileverà automaticamente tali modifiche. Per aggiornare il meta catalogo con le modifiche effettuate sul database, sarà necessario eseguire un aggiornamento.

Una volta selezionata la card “Aggiorna”, verrà mostrata la data-table-bottom-bar con la lista dei meta cataloghi, dalla quale selezioneremo quello da aggiornare. Dopo la selezione, tramite una chiamata al backend aggiorneremo i valori degli schemas e delle tabelle del meta catalogo. In presenza di conflitti, verremo notificati attraverso un dialog (come già visto nella modifica) che ci informerà dei conflitti risolti.

Eliminare un meta catalogo

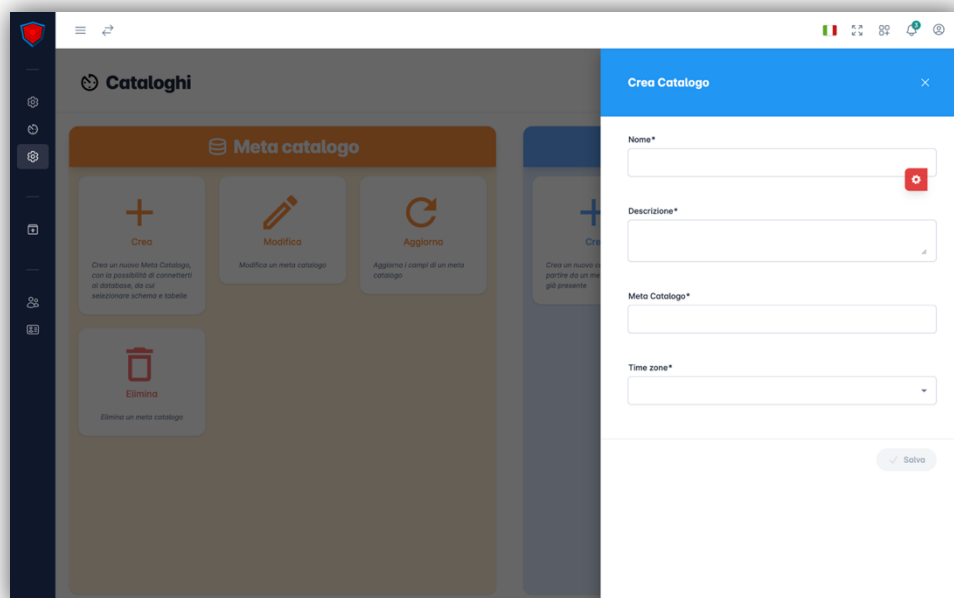
Selezionando la card per l’eliminazione, potremo decidere quale meta catalogo eliminare attraverso la data-table-bottom-bar. Come per le altre pagine, l’eliminazione avverrà tramite una chiamata DELETE all’API del backend.

Cataloghi

Nel lato destro della pagina troveremo tre card, ciascuna delle quali rappresenta un’interazione possibile con i cataloghi.

Creare un catalogo

La prima interazione disponibile è la creazione di un catalogo.

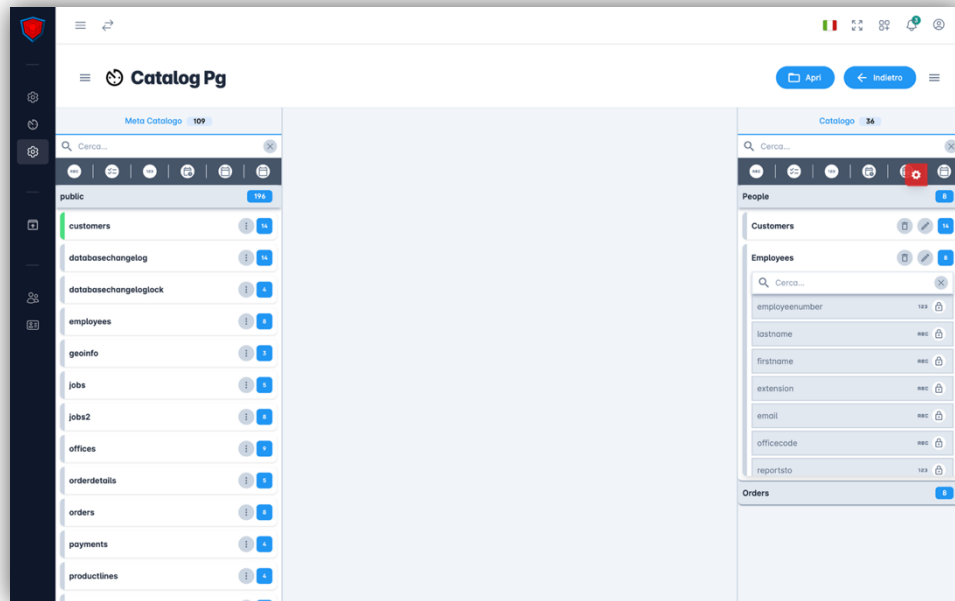


Una volta selezionata la card corrispondente, si aprirà un drawer dal lato destro della pagina, all’interno del quale troveremo un form per inserire le informazioni necessarie alla creazione. Dopo

aver compilato il form, si procederà con la creazione del catalogo tramite una chiamata all'API dedicata nel backend. In caso di successo o fallimento, verremo notificati tramite un alert.

Visualizzare un catalogo

Selezionando la card relativa alla visualizzazione, si aprirà la data-table-bottom-bar contenente la lista dei cataloghi da cui selezionare quello desiderato.



Dopo la selezione, verremo portati nella pagina di visualizzazione, composta da due drawer:

- A sinistra: la lista dei meta cataloghi associati al catalogo
- A destra: la lista delle viste associate al catalogo selezionato

Entrambe le liste vengono ottenute tramite una chiamata GET al backend e successivamente filtrate e riordinate, per consentire una visualizzazione corretta all'interno del componente `AccordionNavigator`.

A differenza del precedente utilizzo, qui l'accordion sarà mostrato con tutti i suoi elementi:

- Titolo
- Barra di ricerca
- Filtri
- Category
- Folder
- Folder menu
- Folder options
- Items
- Informazioni sugli items

Le folder rappresentano gli schemas salvati nel meta catalogo e gli items rappresentano le tabelle. Le folder vengono raggruppate all'interno delle category, in base alla categoria associata allo schema.

Eliminare un catalogo

L'ultima interazione disponibile riguarda l'eliminazione di un catalogo.

Una volta selezionata la card, si aprirà la data-table-bottom-bar con la lista dei cataloghi, dalla quale selezioneremo quello da eliminare. Dopo la selezione, l'eliminazione avverrà tramite una chiamata DELETE all'API del backend, come nel caso dell'eliminazione di un meta catalogo.

Conclusione

Durante questi due anni di ITS Angelo Rizzoli ho avuto la possibilità di acquisire nuove competenze nel campo dello sviluppo software.

Lo stage che ho svolto nell'ultima parte del percorso formativo mi ha dato l'opportunità di occuparmi dello sviluppo del progetto KubeX, software dedicato al cliente Telecom, attraverso il framework Angular appartenente a Google.

I miei colleghi, nonostante la distanza, mi hanno prestato aiuto e supporto durante questo periodo, aiutandomi a crescere e apprendere nuove conoscenze, insegnandomi a lavorare in gruppo in un grande progetto dove ci lavorano più sviluppatori.

Lo stage mi ha dato la possibilità di migliorare le mie capacità di analisi e problem solving, indispensabili in un contesto lavorativo.

Posso concludere affermando che l'esperienza appena terminata mi ha fatto crescere e mi ha dato nuovi stimoli per migliorare sempre di più in ambito lavorativo e personale.

Ringraziamenti

Volevo dedicare questo capitolo alle persone che mi hanno supportato e mi sono state vicine durante questo percorso.

Ringrazio i docenti e tutto lo staff dell'ITS Angelo Rizzoli per essermi stati vicini durante questi due anni e per avermi dato l'opportunità di intraprendere questo percorso.

Ringrazio tutti i colleghi di Vantea Smart che mi hanno accolto all'interno della loro azienda, facendomi sentire parte di un team.

Ringrazio Luca per avermi dato la grande opportunità di lavorare all'interno del suo team, per avermi dato fiducia fin dall'inizio e per essermi stato vicino durante tutto il percorso di stage.

Ringrazio Cristian per avermi affiancato durante lo stage, per essermi stato vicino e per avermi insegnato ogni giorno cose nuove.

Ringrazio tutti i miei amici e i compagni di corso per il bel tempo che abbiamo passato insieme, all'interno e all'esterno dell'ITS.

Infine, ringrazio tutta la mia famiglia per avermi dato la possibilità di studiare all'ITS, per essermi stata vicina durante tutto questo percorso e per avermi supportato nei momenti di bisogno.

Sitografia

Vantea

<https://www.vantea.com/>

Angular

<https://angular.dev/>

Linux

<https://www.linux.it/>

Webstorm

<https://www.jetbrains.com/webstorm/>

Open Vpn

<https://openvpn.net/>

Jira

<https://www.atlassian.com/software/jira>

Confluence

<https://www.atlassian.com/software/confluence>

BitBucket

<https://bitbucket.org/product/>

Microsoft Teams

<https://teams.live.com/free>

Corso Angular

https://youtu.be/-NewNF-rA5c?si=6z7_X17b5re0w7iE